

Universität Siegen
Naturwissenschaftlich-Technische Fakultät
Department Elektrotechnik und Informatik
Lehrstuhl für Digital Integrated Systems

Master Thesis

Evaluation model of a parametrizable ReRAM core in an FPGA architecture

by

Hrishikesh Balkrishna Karande

Matriculation number

1701995

Examiner

Prof. Dr.-Ing. Michael Wahl

2nd Examiner

Prof. Dr.-Ing. Bing Li

Date of delivery

03.11.2025

Declaration of Authorship

I certify that I have prepared this written work independently and have not used any aids other than those specified. I have clearly identified all passages that are taken from other works in terms of wording or meaning (including translations) as borrowed material in each individual case, stating the exact source (including the World Wide Web as well as generative AI and other electronic data collections). This also applies to attached drawings, pictorial representations, sketches and the like. I acknowledge that the proven omission of the indication of origin will be regarded as attempted deception.

I confirm that the content of the electronic version is the same as the printed version.

(Date)

(Signature)

Foreword

I would like to take this opportunity to express my sincere gratitude to the people who have supported me throughout this Master Thesis.

First and foremost, I am deeply indebted to my supervisor, Prof. Dr.-Ing. Michael Wahl. His guidance, insightful feedback, and willingness to make time for our weekly meetings were invaluable. His expertise and consistent support provided a clear direction and helped me navigate the challenges of this research.

I also extend my heartfelt thanks to my family. Their unwavering encouragement, patience, and belief in me have been my foundation. Without their support, this journey would not have been possible.

Finally, my gratitude goes to my friends- Sumukh Marathe, Isa Hassan, Haris Mujeeb, Usama Junaid and Leena Kashyap who provided a much-needed source of motivation, perspective, and timely distractions.

Hrishikesh Balkrishna Karande

This thesis presents the design, implementation, and FPGA-based emulation of a Resistive Random Access Memory (ReRAM) model, targeting the Xilinx Zybo Z7-7020 development board. A behavioral SystemVerilog model of a single ReRAM cell is developed, incorporating key device characteristics such as Low Resistance State (LRS), High Resistance State (HRS), write delays, and endurance limits. The model is extended into a scalable ReRAM core array for system-level evaluation. The design is synthesized and deployed on the FPGA, with an AXI4-Lite interface enabling communication with the on-chip ARM processing system for control, data exchange, and hardware-software validation. This platform provides a foundation for exploring ReRAM-based accelerators and processing-in-memory systems. The work emphasizes reproducibility, with all design files, testbenches, and documentation structured for seamless replication and further research. The complete project, including source code and comprehensive documentation, is available in this GitHub repository: https://github.com/hrishikeshkarande/CNN-ReRAM_on_FPGA.

Contents

Declaration of Authorship	ii
Foreword	iii
Abstract	iv
1 Introduction	1
1.1 Background & Motivation	1
1.2 Problem Statement	1
1.3 Research Objectives	1
1.4 Scope of Work	2
1.5 Thesis Structure	3
2 Literature Review	4
2.1 Overview of Non-Volatile Memories	4
2.1.1 Flash Memory	4
2.1.2 Magnetoresistive Random Access Memory (MRAM)	4
2.1.3 Phase-Change Memory (PCM)	5
2.1.4 Ferroelectric Random Access Memory (FeRAM)	5
2.1.5 Resistive Random Access Memory (ReRAM)	5
2.1.6 Comparative Summary	6
2.2 Physics and Switching Mechanisms of ReRAM Devices	6
2.2.1 Device Structure and Basic Operation	6
2.2.2 Filamentary Resistive Switching Mechanism	7
2.2.3 Interfacial Resistive Switching Mechanism	8
2.2.4 Electrical Characteristics of Resistive Switching	8
2.2.5 Summary of Physical Operation	9
2.3 ReRAM Device Modeling Approaches and FPGA-Based Emulation	9
2.3.1 Hierarchy of ReRAM Modeling	10
2.3.2 Compact Modeling and Its Limitations	11
2.3.3 Behavioral Modeling for FPGA-Based Prototyping	12
2.3.4 FPGA-Based ReRAM Emulation	13
2.3.5 Advantages of FPGA-Based ReRAM Modeling	14
2.3.6 Summary	14
2.4 Use Cases and Applications of ReRAM in Processing-in-Memory and Neuromorphic Systems	14
2.4.1 Motivation: The Memory Wall and PIM Concept	14
2.4.2 Analog Computing with ReRAM Crossbar Arrays	15
2.4.3 ReRAM for Digital and Hybrid PIM Architectures	16
2.4.4 Neuromorphic Computing and Synaptic Emulation	16
2.4.5 ReRAM in Edge AI and Embedded Systems	16
2.4.6 FPGA-Based Emulation of ReRAM PIM Systems	17

2.4.7	Summary	17
2.5	Summary of Literature Review and Research Gap	17
2.5.1	Summary of Key Findings	17
2.5.2	Identified Research Gaps	18
2.5.3	Research Direction of This Thesis	19
2.5.4	Summary	19
3	System Architecture	20
3.1	Overview of the System	20
3.2	Hardware Architecture	21
3.2.1	Hardware Components	21
3.2.2	FPGA Implementation Strategy and Resource Profile	22
3.2.3	System Interconnect	23
3.3	Software Architecture	23
3.3.1	Software Components	23
3.3.2	Software–Hardware Communication Flow	24
3.3.3	Development Environment	24
3.4	Integration and System Workflow	25
3.4.1	Advantages of the Architecture	25
3.4.2	Summary	25
4	Implementation	26
4.1	Introduction	26
4.2	L0: The ReRAM Cell (<code>reram_cell_simple</code>)	26
4.2.1	Purpose:	26
4.2.2	Key Features:	27
4.2.3	State Machine Implementation:	29
4.3	L1: The ReRAM Word Array (<code>reram_word_array_simple</code>)	29
4.3.1	Purpose:	29
4.3.2	Architecture:	29
4.3.3	Functional Analysis:	29
4.4	L2: The Memory Controller (<code>reram_memory_controller_blocking</code>)	31
4.4.1	Purpose:	31
4.4.2	Architecture:	31
4.4.3	Functional Analysis:	31
4.5	L3: The AXI-Lite Wrapper (<code>reram_axilite_wrapper</code>)	33
4.5.1	Purpose:	33
4.5.2	Address Mapping:	33
4.5.3	Transaction Translation:	33
4.6	Conclusion	35
5	Experimental Setup and Implementation	36
5.1	Introduction	36

5.2	Design and Implementation Methodology	36
5.2.1	Vivado Project Flow	36
5.3	Functional Verification (Simulation)	37
5.3.1	L0: Cell-Level Testbench (tb_reram_cell_simple)	37
5.3.2	L1: Word-Array-Level Testbench (tb_reram_word_array_simple)	38
5.3.3	L2: Controller-Level Testbench (tb_reram_memory_controller_blocking)	39
5.3.4	L3: AXI-Wrapper-Level Testbench (tb_reram_axilite_wrapper)	40
5.3.5	L0 Cell-Level Testbench Analysis	41
5.3.6	L3 AXI-Level Testbench	42
5.3.7	Simulation Results and Waveform Analysis	42
5.4	Implementation Results	43
5.4.1	RTL and Synthesized Schematics	44
5.4.2	Resource Utilization	44
5.4.3	Power Utilization	44
5.5	Reconfiguration to check parameterizability and scalability	47
5.5.1	Power Utilization Comparison	48
5.6	Software Setup (Xilinx SDK / Vitis)	48
5.7	Chapter Summary	50
6	Conclusion	51
6.1	Future Work	51
6.2	Resources Used	52
6.2.1	AI-Powered Development Tools	52
6.2.2	Textbooks and Academic Literature	52
	Appendix	54
	References	74

List of Figures

Abb. 1	ReRAM Switching Mechanism (HRS vs. LRS)	8
Abb. 2	ReRAM Modeling Hierarchy Pyramid	10
Abb. 3	ReRAM Crossbar Array for PIM	15
Abb. 4	Zybo Z7 7020 FPGA Development Board	20
Abb. 5	Overall System Architecture	21
Abb. 6	Design Hierarchy	27
Abb. 7	Logical dataflow for L0 ReRAM Cell	28
Abb. 8	Logical dataflow for L1 ReRAM Word Array	30
Abb. 9	Logical dataflow for L2 Memory Controller	32
Abb. 10	Logical dataflow for L3 AXI-Lite Wrapper	34
Abb. 11	Vivado system block design (Zynq PS and ReRAM IP)	37
Abb. 12	Waveform for L0 ReRAM Cell (reram_cell_simple)	42
Abb. 13	Waveform for L3 AXI blocking write transaction	43

Abb. 14	Elaborated RTL schematic of the <code>reram_axilite_wrapper</code>	45
Abb. 15	Synthesized schematic showing the design mapped to logic gates.	46
Abb. 16	Power report summary from the Vivado Power Analyzer.	46
Abb. 17	Vivado Implementation Results (Configuration 2)	47
Abb. 18	SDK/Vitis Hardware-Software Validation	49

Listings

1	High-Level Behavioral Model of Write Operation	12
2	High-Level Functions for ReRAM Access	23
3	Simulation console output for AXI testbench.	42
4	RTL for <code>reram_cell_simple.sv</code>	54
5	RTL for <code>reram_word_array_simple.sv</code>	59
6	RTL for <code>reram_memory_controller_blocking.sv</code>	60
7	RTL for <code>reram_axilite_wrapper.sv</code>	63
8	L0 Testbench for <code>reram_cell_simple</code>	66
9	L1 Testbench for <code>reram_word_array_simple</code>	68
10	L2 Testbench for <code>reram_memory_controller_blocking</code>	69
11	L3 Testbench for <code>reram_axilite_wrapper</code>	71
12	Bare-metal C code for testing the ReRAM IP.	73

List of Tables

Tab. 1	Comparison of Non-Volatile Memory Technologies	6
Tab. 2	Operation Modes and Electrical Characteristics of a Bipolar ReRAM Cell	9
Tab. 3	Comparison of ReRAM Modeling Techniques for System-Level Evaluation	14
Tab. 4	ReRAM Memory Control Unit Register Map (Implemented)	22
Tab. 5	Post-Synthesis Resource Utilization	44
Tab. 6	Resource Utilization Comparison	47
Tab. 7	Power Utilization Comparison	48

1. Introduction

1.1. Background & Motivation

Resistive Random Access Memory (ReRAM) has emerged as a promising non-volatile memory technology due to its scalability, low power consumption, and potential for in-memory computing. Unlike conventional charge-based memories, ReRAM stores information by switching between distinct resistance states—Low Resistance State (LRS) and High Resistance State (HRS) through the application of electrical stimuli. This property enables not only high-density storage but also the possibility of embedding computation within the memory array, significantly reducing data movement overhead.

In recent years, the growing computational demands of data-intensive applications, particularly in artificial intelligence (AI) and machine learning, have intensified interest in hardware accelerators that leverage emerging memory technologies. ReRAM’s ability to combine storage and computation in a single structure positions it as a candidate for efficient implementations of neural networks and other parallel processing workloads.

However, physical ReRAM devices are still in active development and not widely accessible for experimental research. FPGA-based emulation provides a practical solution, allowing researchers to model ReRAM behavior, test architectures, and validate algorithms before fabrication. By implementing a configurable, cycle-accurate behavioral model of ReRAM on the Xilinx Zybo Z7-7020 board, this work aims to bridge the gap between device-level models and system-level applications. Such an emulation platform supports reproducible experimentation, facilitates performance evaluation, and paves the way for integrating ReRAM into real-time AI accelerators.

1.2. Problem Statement

The rapid growth of data-intensive applications, particularly in artificial intelligence and machine learning, has created a pressing need for high-performance and energy-efficient computing architectures. Traditional von Neumann systems suffer from the “memory wall” bottleneck, where the cost of moving data between memory and processing units dominates execution time and power consumption.

Emerging non-volatile memory technologies such as Resistive Random Access Memory (ReRAM) offer the potential to overcome these limitations by enabling in-memory computation. However, physical ReRAM devices are not yet widely available for large-scale experimentation, and their fabrication costs can be prohibitive for academic research.

This creates a clear need for a flexible, FPGA-based emulation platform that can accurately model the behavior of ReRAM cells and arrays, including their timing characteristics, endurance limits, and data storage capabilities. Such a platform should support integration with processor systems, enabling real-world software-hardware co-design experiments and facilitating the evaluation of ReRAM-based accelerators for applications such as Convolutional Neural Networks (CNNs).

1.3. Research Objectives

The primary objective of this thesis is to design, implement, and evaluate a behavioral emulation of ReRAM on the Xilinx Zybo Z7-7020 FPGA platform. The specific objectives are as follows:

1. **ReRAM Cell Modeling:** Develop a SystemVerilog-based behavioral model of a single ReRAM cell, incorporating resistive state switching, write delays, endurance limitations, and data storage capability.
2. **ReRAM Core Design:** Extend the single-cell model into a scalable ReRAM core array, supporting addressable read/write operations and enabling system-level integration.
3. **FPGA Integration:** Implement the ReRAM core on the Zybo Z7-7020 board and interface it with the on-chip ARM processing system using an AXI4-Lite communication protocol.
4. **Hardware-Software Validation:** Demonstrate the functional correctness of the integrated system by developing a bare-metal C application for the ARM processor, capable of controlling and verifying the ReRAM core's behavior through AXI-Lite transactions.
5. **Reproducible Research:** Document the design methodology, simulation results, FPGA synthesis steps, and software integration to ensure that the work can be replicated and extended by other researchers.

1.4. Scope of Work

This thesis focuses on the FPGA-based behavioral emulation of Resistive Random Access Memory (ReRAM) for research and prototyping purposes. The work is limited to modeling the functional and timing characteristics of ReRAM at a behavioral level and does not include transistor-level device simulations or fabrication processes. The key aspects of the scope are as follows:

- **Behavioral Modeling:** Development of a cycle-accurate, high-level SystemVerilog model of a single ReRAM cell and its extension into a multi-cell core array.
- **FPGA Implementation:** Deployment of the ReRAM core on the Xilinx Zybo Z7-7020 platform, including synthesis, place-and-route, and hardware validation.
- **Processor Integration:** Implementation of an AXI4-Lite interface for communication between the FPGA fabric and the ARM Cortex-A9 processing system on the Zybo board.
- **Functional Demonstration:** Verification of the hardware-software co-design by executing a bare-metal C application on the ARM processor to perform read/write operations and retrieve status from the ReRAM IP core, validating the AXI4-Lite communication channel and the core's functionality.
- **Evaluation and Documentation:** Analysis of functional correctness, resource utilization, and performance, along with complete documentation to ensure reproducibility.

The scope explicitly excludes:

- Physical fabrication or testing of actual ReRAM devices.
- Detailed electrical or material level modeling of resistive switching phenomena.
- The design and implementation of any application-level accelerators, such as Convolutional Neural Networks (CNNs), which would utilize the ReRAM core. This is left for future work.

1.5. Thesis Structure

This thesis is organized as follows:

- **Chapter 2 (Literature Review)** provides a comprehensive review of non-volatile memories, the physics of ReRAM, existing modeling approaches, and the use of ReRAM in in-memory computing systems, identifying the research gap this work addresses.
- **Chapter 3 (System Architecture)** details the high-level design of the emulation platform, covering the hardware-software co-design, the modular hierarchy, and the AXI4-Lite communication interface.
- **Chapter 4 (Implementation)** describes the hierarchical, bottom-up implementation of the ReRAM core in SystemVerilog, from the L0 cell to the L3 AXI-Lite wrapper.
- **Chapter 5 (Experimental Setup and Implementation)** presents the functional verification, synthesis, and on-chip validation of the design, including resource analysis and hardware-software co-design testing.
- **Chapter 6 (Conclusion)** summarizes the contributions of this work, discusses the results, and proposes directions for future research.
- **Appendix** provides the complete source code for the RTL design, verification testbenches, and the C-based software application.

2. Literature Review

2.1. Overview of Non-Volatile Memories

Non-volatile memory (NVM) technologies play a critical role in modern computing systems by retaining information even when power is removed. They form the foundation for persistent data storage in applications ranging from consumer electronics and data centers to embedded and edge devices. Over the last two decades, several emerging NVM technologies have been investigated to overcome the scaling limitations of traditional charge-based memories such as Flash. Among these, Magnetoresistive Random Access Memory (MRAM), Phase-Change Memory (PCM), Ferroelectric Random Access Memory (FeRAM), and Resistive Random Access Memory (ReRAM) have gained significant attention for their potential to deliver high density, fast access, and low-power operation [24].

Each of these technologies is based on distinct physical principles that govern how information is stored and retrieved. The following subsections provide an overview of their operation mechanisms, advantages, and limitations in comparison to conventional Flash memory.

2.1.1. Flash Memory

Flash memory has been the dominant non-volatile storage technology for decades due to its scalability, cost efficiency, and mature manufacturing ecosystem. It stores data by trapping electric charge in a floating-gate or charge-trap transistor structure. Information is represented by the amount of charge stored, which shifts the transistor threshold voltage and enables binary or multi-level storage.

Two main types exist: NOR Flash, optimized for random access and code execution, and NAND Flash, optimized for dense storage and sequential access. NAND Flash powers most solid-state drives (SSDs) and embedded storage solutions. However, as device geometries scale below 20 nm, charge leakage, endurance degradation, and slow program/erase cycles have become major challenges. Furthermore, Flash memories typically require high programming voltages (10–20 V) and suffer from limited endurance (10^3 – 10^5 cycles) and high latency during erase operations. These limitations motivate the exploration of alternative, non-charge-based memory mechanisms.

2.1.2. Magnetoresistive Random Access Memory (MRAM)

MRAM stores data through magnetic polarization rather than electric charge. A typical MRAM cell consists of a magnetic tunnel junction (MTJ) — two ferromagnetic layers separated by a thin insulating barrier (often MgO). One layer has a fixed magnetic orientation, while the other is free to switch its magnetization. Depending on whether the layers are aligned parallel or antiparallel, the junction exhibits low or high electrical resistance, corresponding to logic “0” and “1” [11].

Earlier generations used field-induced magnetic switching, but modern variants employ Spin-Transfer Torque (STT-MRAM) or Spin-Orbit Torque (SOT-MRAM), which enable lower switching energy and better scalability. MRAM offers excellent endurance ($> 10^{15}$ cycles) and nanosecond-scale access times, rivaling SRAM while providing non-volatility. Its main challenges are high write current requirements and complex fabrication, which increase cost and limit density compared to NAND Flash. Nevertheless, STT-MRAM is now commercially adopted in cache and embedded applications, bridging the gap between volatile and non-volatile storage tiers [11].

2.1.3. Phase-Change Memory (PCM)

Phase-Change Memory exploits the large resistance contrast between the amorphous and crystalline phases of chalcogenide materials, typically Ge Sb Te (GST). Applying electrical pulses of varying amplitude and duration controls the local temperature of the material, inducing reversible phase transitions that represent binary states [21].

PCM offers fast switching (tens of nanoseconds), multi-bit storage capability, and good retention (> 10 years). It also supports in-place updates, eliminating the erase-before-write constraint of Flash. However, high programming current and thermal crosstalk between cells limit scalability. PCM endurance (10^7 – 10^8 cycles) is superior to Flash but inferior to MRAM, and device variability remains a key research issue. Intel’s 3D XPoint™ technology, although partially proprietary, is widely believed to be PCM-based and demonstrates its viability for persistent memory systems [21].

2.1.4. Ferroelectric Random Access Memory (FeRAM)

FeRAM stores data using the polarization state of a ferroelectric material (commonly Pb(Zr,Ti)O or HfO -based thin films) sandwiched between electrodes. When an electric field is applied, the dipoles within the material align in one of two stable polarization directions, corresponding to logic “0” and “1”. The state can be read by measuring the polarization charge or current pulse generated when switching [23].

FeRAM features very low power consumption, fast access times (tens of nanoseconds), and high endurance (10^{10} – 10^{14} cycles). It has been used in embedded microcontrollers and low-power sensor applications. The primary limitation is low storage density, as the ferroelectric capacitor requires relatively large cell area and is incompatible with standard CMOS scaling. Recent advances in HfO -based FeRAM have revived interest by improving CMOS compatibility and offering potential for 3D integration [3].

2.1.5. Resistive Random Access Memory (ReRAM)

Resistive Random Access Memory (ReRAM), also known as memristive memory, operates by modulating the electrical resistance of a thin dielectric film sandwiched between two electrodes. When a suitable voltage is applied, conductive filaments or interfacial redox reactions alter the device’s resistive state. Switching between the Low-Resistance State (LRS) and High-Resistance State (HRS) encodes binary information [1].

The underlying mechanisms vary depending on the material system — for instance, oxygen-vacancy migration in HfO /TiO , cation migration in Ag/Ta O , or valence-change effects in perovskite oxides [28]. ReRAM offers low operating voltage (< 2 V), fast switching (< 10 ns), scalability beyond 10 nm, and 3D stackability, making it one of the most promising candidates for next-generation non-volatile memory and in-memory computing [13].

A notable advantage of ReRAM is its analog tunability, where intermediate resistance states can be programmed. This property enables the implementation of multilevel storage and analog computing, particularly processing-in-memory (PIM) and neuromorphic computing paradigms. However, ReRAM still faces challenges such as device variability, endurance degradation, and retention instability. Research efforts are increasingly focusing on behavioral modeling, FPGA emulation, and

algorithm–hardware co-design to evaluate ReRAM systems at architectural level before physical fabrication [4], [17].

2.1.6. Comparative Summary

Table 1: Comparison of Non-Volatile Memory Technologies

Memory Type	Mechanism	R/W Speed	Endurance	Density	Maturity
Flash	Charge storage	μs – ms	10^3 – 10^5	High	Commercial
STT-MRAM	Magnetic polarization	ns	10^{15}	Medium	Commercial
PCM	Phase transition	ns– μs	10^7 – 10^8	High	Prototypical
FeRAM	Polarization	ns	10^{10} – 10^{14}	Low– Medium	Mature (embedded)
ReRAM	Resistive switching	<10 ns	10^6 – 10^9	Very High	Emerging

ReRAM stands out for its combination of speed, scalability, and structural simplicity, which enable its use not only as a memory but also as a computational primitive for energy-efficient architectures.

In summary, while Flash memory remains dominant for mass storage, its scaling challenges necessitate alternative solutions. MRAM and FeRAM offer excellent endurance and speed but are limited by cost and density. PCM provides multi-bit capability but suffers from thermal constraints. ReRAM, owing to its filamentary resistive switching and CMOS compatibility, represents the most versatile candidate for future universal memory and processing-in-memory systems. Consequently, this thesis focuses on the FPGA-based behavioral modeling and evaluation of ReRAM architectures, enabling early-stage system-level exploration before large-scale hardware realization.

2.2. Physics and Switching Mechanisms of ReRAM Devices

Resistive Random Access Memory (ReRAM) devices are a class of non-volatile memories that store data by exploiting a change in the electrical resistance of a dielectric material. This change—known as resistive switching (RS)—is reversible and can be electrically triggered by applying appropriate voltage pulses. Depending on the polarity and nature of the voltage required for switching, ReRAM devices are broadly classified as unipolar or bipolar.

The physical origin of resistive switching is closely related to the formation and rupture of conductive filaments or modulation of the interface barrier within the oxide layer. These mechanisms are material-dependent but share the same conceptual basis: an applied voltage drives ionic motion or redox reactions, which locally alter the conductivity of the device, thereby encoding a logical state.

2.2.1. Device Structure and Basic Operation

A typical ReRAM device has a simple metal–insulator–metal (MIM) structure, consisting of:

- Top electrode (TE): Often inert (Pt, Au) or active (Ag, Cu) depending on the switching mechanism.

- Switching layer: A thin dielectric or oxide film such as HfO_2 , TiO_2 , Al_2O_3 , or Ta_2O_5 .
- Bottom electrode (BE): Usually inert (TiN , Pt) to provide a stable reference.

This simple vertical stack allows for CMOS compatibility and 3D stacking, making ReRAM an attractive candidate for high-density integration [1], [22].

At the microscopic level, the switching layer contains mobile ions or oxygen vacancies that can migrate under an electric field. The distribution of these defects determines whether the device is in a Low Resistance State (LRS)—conductive path formed—or a High Resistance State (HRS)—path ruptured or depleted.

Before normal operation, most devices require an initial electroforming step, during which a high voltage creates a soft conductive filament through the dielectric. After forming, subsequent switching occurs at much lower voltages and can be repeatedly cycled between LRS and HRS [10].

2.2.2. Filamentary Resistive Switching Mechanism

In filamentary ReRAM, resistance change is governed by the creation and rupture of localized conductive filaments (CFs). These filaments are formed by either:

1. Cation Migration (Electrochemical Metallization, ECM)

- Active metal ions (Ag^+ , Cu^+) dissolve into the dielectric under positive bias at the top electrode.
- The cations drift through the oxide and are reduced at the bottom electrode, depositing metallic atoms that grow upward as a filament.
- Reversing the voltage polarity dissolves the filament, returning the device to HRS (High Resistance State).
- **Example Systems:** $\text{Ag}/\text{Ta}_2\text{O}_5/\text{Pt}$ or $\text{Cu}/\text{SiO}_2/\text{Pt}$. [14], [19].

2. Anion (Oxygen Vacancy) Migration (Valence Change Mechanism, VCM)

- Oxygen vacancies act as donors, increasing local conductivity in transition metal oxides such as HfO_2 , TiO_2 , or NiO .
- Applying a positive bias drives oxygen anions toward the anode, leaving behind oxygen vacancies that cluster into a conductive path.
- Reversing the bias moves oxygen back into the filament region, oxidizing it and rupturing the conductive path [31], [10].

The SET operation as shown in Figure 1 corresponds to forming or reestablishing the filament (switching from HRS $\rightarrow$ LRS), while the RESET operation ruptures the filament (LRS $\rightarrow$ HRS).

This switching behavior is highly nonlinear, and the device I–V curve typically exhibits a “pinched hysteresis loop”—a signature characteristic of memristive systems [9].

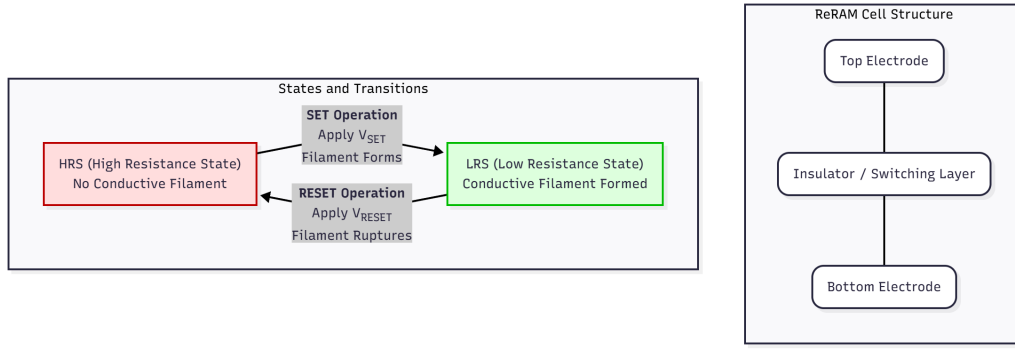


Figure 1: Conceptual diagram of the ReRAM switching mechanism. The device transitions between a High Resistance State (HRS), where the conductive filament is broken, and a Low Resistance State (LRS), where the filament is formed, by applying SET and RESET voltages.

2.2.3. Interfacial Resistive Switching Mechanism

In contrast to filamentary mechanisms, interfacial ReRAM operates by modifying the Schottky barrier or interfacial redox layer at the metal/oxide interface. Here, resistance modulation occurs uniformly across the contact area rather than through discrete filaments.

In such devices, an applied bias alters the oxygen vacancy concentration near the interface or changes the oxidation state of the electrode, modifying the tunneling barrier height and width [22].

For instance, in TiO /Pt and SrTiO -based ReRAM, the drift of oxygen ions at the interface leads to controllable changes in barrier height, resulting in bipolar switching behavior.

Interfacial switching typically offers better uniformity and multi-level resistance states, but slower switching speeds compared to filamentary ReRAM. It is also more suitable for analog computing and synaptic emulation, where gradual conductance modulation is desired [33].

2.2.4. Electrical Characteristics of Resistive Switching

The switching sequence can be summarized as follows:

1. Electroforming: A high voltage (3–5 V) initiates soft dielectric breakdown, creating a conductive path through defect generation. This step is performed once.
2. SET Process (Write ‘1’): A moderate positive bias (1–2 V) triggers oxygen vacancy drift or cation reduction, forming a conductive filament. The device switches from HRS to LRS, allowing higher current flow.
 - Physical process: reduction reaction / filament growth.
3. RESET Process (Write ‘0’): Reversing the voltage polarity (–1 to –2 V) or applying a high compliance current causes filament dissolution or oxidation, restoring HRS.
 - Physical process: oxidation reaction / filament rupture.
4. READ Operation: A small sensing voltage (typically <0.2 V) is applied to measure the current.
 - The read voltage is chosen low enough to avoid altering the device state.

- The measured current distinguishes between LRS and HRS.

These operations are summarized in Table 2.

Table 2: Operation Modes and Electrical Characteristics of a Bipolar ReRAM Cell

Operation	Voltage Range	Direction	Physical Mechanism	Logical State Change
Electroforming	3–5 V	+	Defect creation / soft breakdown	Initial HRS → LRS
SET (Write 1)	1–2 V	+	Filament formation (reduction)	HRS → LRS
RESET (Write 0)	1–2 V	–	Filament rupture (oxidation)	LRS → HRS
READ	<0.2 V	+	Non-destructive current sensing	LRS or HRS retained

ReRAM’s endurance and variability depend heavily on filament geometry, material composition, and compliance current used during SET operation.

To ensure reproducibility, practical systems incorporate current compliance circuits or selector transistors that limit write current and prevent permanent breakdown [7].

2.2.5. Summary of Physical Operation

In essence, the resistive switching process in ReRAM is a manifestation of field-driven ion migration and redox dynamics.

- The SET transition corresponds to the formation of a conductive filament or interfacial reduction, converting the device into a low-resistance state.
- The RESET transition involves rupture or oxidation of this path, restoring high resistance.
- The READ process is a low-voltage measurement of resistance, ensuring non-destructive state detection.

These simple yet powerful mechanisms enable ReRAM to combine non-volatility, fast switching, analog tunability, and CMOS compatibility, making it an ideal candidate for in-memory computation and neuromorphic architectures [13], [32], [24], [26].

2.3. ReRAM Device Modeling Approaches and FPGA-Based Emulation

The resistive switching phenomena in ReRAM devices are governed by complex, nonlinear electrochemical and thermodynamic processes involving ion migration, charge trapping, and redox reactions within nanometer-scale oxide layers. Accurately capturing these mechanisms across scales — from materials to system level is essential for predicting performance, endurance, and variability. However, direct physical simulation using finite-element or ab-initio models is computationally intensive and unsuitable for large-scale circuit or architectural evaluation. Consequently, researchers have developed a hierarchy of modeling approaches to represent ReRAM behavior at different abstraction levels

— physical, compact, behavioral, and system-level — each offering a trade-off between accuracy and computational efficiency [12], [2], [7].

2.3.1. Hierarchy of ReRAM Modeling

ReRAM modeling can be broadly categorized into the following levels (also shown in Figure 2):

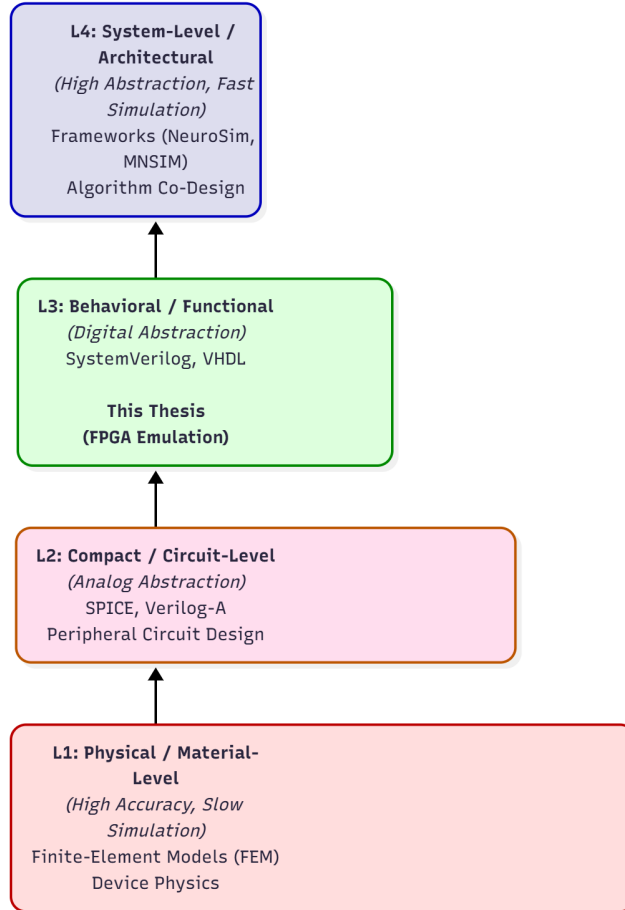


Figure 2: The ReRAM modeling hierarchy, showing the trade-off between simulation speed and physical accuracy. This thesis operates at the behavioral/functional level, providing a fast, synthesizable model for system-level emulation.

1. Physical and Material-Level Models:

- Based on drift–diffusion and thermochemical equations that describe the movement of ions and vacancies, Joule heating, and redox kinetics within the oxide [12].
- These models often solve coupled partial differential equations (PDEs) to simulate filament evolution and heat transport, using tools such as COMSOL Multiphysics or finite-element solvers.
- Although highly accurate, they are computationally prohibitive for circuit-scale simulations and are mainly used for understanding device physics or guiding material optimization.

2. Compact (Circuit-Level) Models:

- Compact models abstract the device into a set of state equations and I–V relationships, enabling integration with SPICE or Verilog-A for circuit simulation [15], [16].
- Popular models include:
 - The Linear Ion Drift Model (Strukov et al., 2008), representing the memristor as a variable resistor whose conductance changes linearly with charge flow.
 - The Nonlinear Ion Drift Model, which incorporates exponential field dependence for more realistic switching behavior.
 - The TEAM (Threshold Adaptive Memristor Model) [15] and VTEAM (Voltage Threshold Adaptive Model) [16], which introduce voltage-dependent thresholds and nonlinear state dynamics, allowing for accurate representation of set/reset asymmetry and stochastic effects.
- Compact models strike a balance between physical accuracy and computational speed, making them suitable for evaluating small ReRAM arrays and peripheral circuits such as sense amplifiers and selectors.

3. Behavioral (Functional) Models:

- These models abstract away the detailed physics and represent ReRAM behavior through state machines or conditional logic that capture key observable characteristics:
 - Two resistive states (LRS/HRS)
 - Threshold-based switching
 - Endurance degradation
 - Write delay and retention behavior
- Behavioral models are typically implemented in SystemVerilog, VHDL, or SystemC, and are ideal for hardware-in-the-loop simulation and FPGA-based prototyping.
- This level of abstraction forms the foundation of the modeling work presented in this thesis.

4. System-Level and Architectural Models:

- At the highest abstraction, ReRAM arrays are modeled as memory elements or crossbar matrices within accelerators for in-memory computing (IMC) or neuromorphic architectures [6].
- Frameworks such as NeuroSim [6] and MNSIM [30] simulate the effect of device non-idealities on network accuracy and energy efficiency, guiding co-design of hardware and algorithms.

2.3.2. Compact Modeling and Its Limitations

Compact models such as **TEAM** and **VTEAM** have become the de facto standard for circuit-level ReRAM evaluation. They capture the internal state variable $x(t)$, representing the normalized conductive filament length, which evolves according to a nonlinear differential equation:

$$\frac{dx}{dt} = f(x, V) \quad (1)$$

where $f(x, V)$ encodes the field- and voltage-dependent drift of ions, and V is the applied bias. The current through the device is modeled as:

$$I = G(x, V) \cdot V \quad (2)$$

where $G(x, V)$ is the dynamic conductance determined by the state variable.

Despite their accuracy, compact models have several drawbacks when scaling to large arrays or system-level evaluation:

- **High computational cost:** Simulating thousands of coupled nonlinear devices in SPICE is prohibitively slow.
- **Limited portability:** Most models are written in Verilog-A or MATLAB, not directly synthesizable for FPGA or digital hardware platforms.
- **Lack of architectural context:** They do not model communication delays, bus interfaces, or parallel data paths essential for embedded and FPGA environments.

For these reasons, compact modeling is often complemented by higher-level behavioral emulation techniques that preserve key device characteristics while being hardware-synthesizable.

2.3.3. Behavioral Modeling for FPGA-Based Prototyping

Behavioral modeling represents ReRAM functionality at a digital abstraction level, typically using HDL constructs such as `always` blocks and finite state machines (FSMs).

This approach models each ReRAM cell as a simple state element that toggles between HRS and LRS based on input voltage threshold or control signals, augmented with timing and endurance constraints.

Typical behavioral parameters include:

- LRS/HRS resistance values (LRS, HRS)
- Set and Reset delay cycles (SET, RESET)
- Endurance limit (Endurance)
- Retention time or failure flag

These parameters can be mapped to hardware counters and registers, making the model synthesizable for FPGA emulation.

A simplified behavioral model can be expressed as:

Listing 1: High-Level Behavioral Model of Write Operation

```

1  if (write_en && !failed) begin
2      if (target_state == LRS) begin
3          // SET operation
4          current_state <= LRS after SET_DELAY;
5          endurance_counter <= endurance_counter + 1;
6      end else begin
7          // RESET operation

```

```
8     current_state <= HRS after RESET_DELAY;  
9     endurance_counter <= endurance_counter + 1;  
10 end  
11 end
```

Such logic captures the essential temporal behavior of resistive switching without solving analog equations. The model is therefore suitable for:

- Rapid cycle-accurate simulation of large ReRAM arrays.
- Integration with AXI4-based processor interfaces.
- System-level validation of ReRAM-based accelerators.

2.3.4. FPGA-Based ReRAM Emulation

Field-Programmable Gate Arrays (FPGAs) provide a flexible platform to emulate the functional and timing behavior of ReRAM arrays at hardware speed.

Since physical ReRAM devices are not yet widely available for academic experimentation, FPGA-based emulation bridges the gap between device models and practical system design [4], [17].

An FPGA-based emulator typically consists of:

1. ReRAM Core: A grid of parameterized ReRAM cells implemented using behavioral HDL models.
2. Control Logic: Finite state machines managing address decoding, write/read scheduling, and endurance tracking.
3. AXI4-Lite or custom bus interface: Enabling communication with embedded processors or external controllers.
4. Monitoring and Logging Units: Recording performance metrics, switching statistics, and data integrity.

This architecture allows researchers to evaluate read/write latency, throughput, and energy consumption in real-time under software control. Moreover, emulation provides a valuable testbed for:

- Processing-in-memory (PIM) algorithms using ReRAM crossbars.
- Error-tolerant neural accelerators with analog-like weight storage.
- Endurance and fault modeling under various workloads.

By parameterizing the delay cycles, endurance limits, and switching probabilities, FPGA-based ReRAM models can mimic device-level stochasticity and performance variability with controllable accuracy.

This makes them ideal for architecture exploration and algorithm co-design, long before hardware fabrication becomes feasible.

Table 3: Comparison of ReRAM Modeling Techniques for System-Level Evaluation

Aspect	Compact/SPICE Models	FPGA Behavioral Models
Abstraction	Analog / device-level	Digital / system-level
Simulation Speed	Slow (μ s per cycle)	Real-time or faster
Scalability	Limited (<100 cells)	Thousands of cells
Hardware Integration	No (software only)	Yes (synthesizable HDL)
Application Scope	Circuit verification	Architecture exploration, PIM
Fidelity	High (physics-accurate)	Moderate (behavioral)

2.3.5. Advantages of FPGA-Based ReRAM Modeling

The FPGA-based approach thus complements traditional modeling by providing real-time emulation and hardware–software co-design capability, which are essential for developing ReRAM-enabled accelerators and neuromorphic systems.

2.3.6. Summary

In summary, ReRAM device modeling spans a continuum from physics-based simulations to real-time behavioral emulation. While compact SPICE-level models are invaluable for understanding device behavior, they are impractical for large-scale integration studies.

FPGA-based behavioral modeling provides a pragmatic compromise, enabling system-level evaluation with configurable parameters that replicate key ReRAM attributes such as switching delay, endurance, and failure behavior.

This thesis builds upon this approach, presenting a parametrizable FPGA ReRAM core that accurately captures the functional and timing characteristics of resistive memory. The design supports AXI4-Lite interfacing, processor integration, and application-level workloads, thereby offering a reproducible and extensible platform for ReRAM research.

2.4. Use Cases and Applications of ReRAM in Processing-in-Memory and Neuromorphic Systems

The intrinsic resistive switching properties of ReRAM make it an attractive candidate not only for dense non-volatile storage but also as a computational primitive.

Unlike conventional memories that merely store bits, ReRAM cells can participate directly in arithmetic operations through their analog conductance modulation.

This enables processing-in-memory (PIM) and neuromorphic computing paradigms, which fundamentally reduce data movement and achieve massive parallelism for data-centric workloads [24], [32].

2.4.1. Motivation: The Memory Wall and PIM Concept

Modern computing systems face the well-known memory wall—a performance bottleneck arising from the separation of compute and memory in von Neumann architectures. In data-intensive tasks such as deep-learning inference, graph analytics, and edge AI, up to 60–80 percent of total energy is spent moving data between the processor and off-chip memory [29].

Processing-in-Memory (PIM) seeks to overcome this by embedding computation directly within or near memory arrays. Since ReRAM can represent data as resistance values and perform current-based summation via Kirchhoff's laws, it offers a natural medium for executing operations such as matrix-vector multiplication (MVM) within the memory fabric itself [20].

2.4.2. Analog Computing with ReRAM Crossbar Arrays

In a typical ReRAM crossbar structure, each memory cell at the intersection of word- and bit-lines stores a conductance G_{ij} proportional to a numerical weight value.

When a vector of input voltages V_j is applied along the word lines, Ohm's and Kirchhoff's laws yield the bit-line currents:

$$I_i = \sum_j G_{ij} V_j \quad (3)$$

which is the fundamental multiply-accumulate (MAC) operation as shown in Figure 3 used in neural-network computations [25], [27].

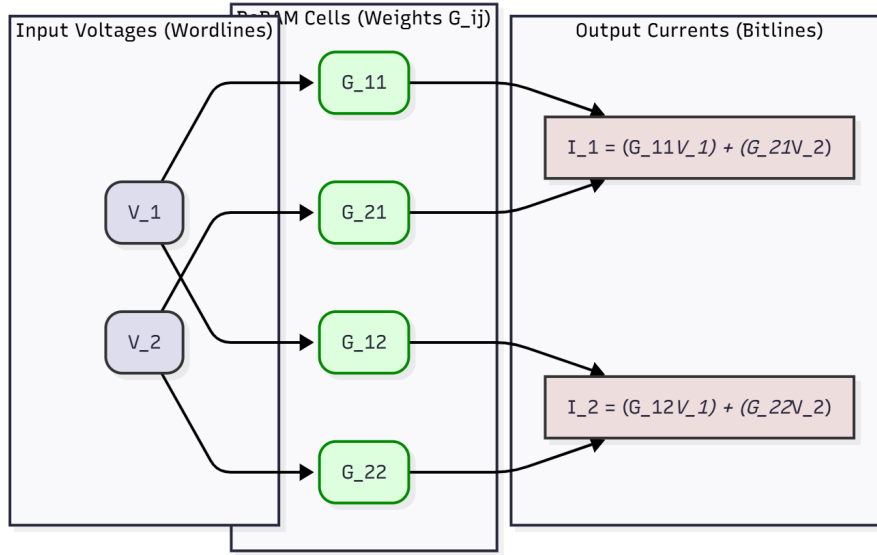


Figure 3: A ReRAM crossbar array performing analog matrix-vector multiplication (MVM). Input voltages (V_j) are applied to the wordlines, and the ReRAM cell's conductance (G_{ij}) modulates the current. The resulting currents are summed along the bitlines (I_i) according to Ohm's Law and Kirchhoff's Law, performing a multiply-accumulate (MAC) operation in-memory.

This in-situ operation eliminates intermediate data transfers and allows massively parallel analog computation. Key advantages include:

- **High parallelism:** Thousands of MACs per clock cycle in one array.
- **Energy efficiency:** Reduces data movement energy by orders of magnitude.
- **Scalability:** Simple MIM cell structure compatible with 3D stacking.

However, analog computation introduces challenges such as device non-linearity, variability, and limited precision.

Techniques such as weight mapping, differential pairs, and digital calibration are commonly used to mitigate these effects [5], [2].

2.4.3. ReRAM for Digital and Hybrid PIM Architectures

Beyond analog MVMS, ReRAM can also support digital PIM architectures, where resistive cells are used as non-volatile lookup elements or configurable logic. In such schemes, logic functions (e.g., AND, OR, XOR) are implemented by exploiting state-dependent current flow across selected cells [26]. Hybrid digital–analog designs combine ReRAM arrays with peripheral CMOS circuits that digitize and accumulate results. For example:

- IMC accelerators such as ISAAC [25] and PRIME [8] employ ReRAM crossbars for analog multiplication, while ADCs/DACs handle conversion and accumulation.
- Edge PIM architectures integrate ReRAM with microcontrollers to enable low-power local inference, suitable for IoT and embedded sensing [32].

FPGA-based emulation, as used in this thesis, provides a practical method to prototype such hybrid systems, allowing real-time verification of interface protocols (AXI4-Lite), latency estimation, and workload validation.

2.4.4. Neuromorphic Computing and Synaptic Emulation

Neuromorphic computing aims to emulate the structure and operation of the human brain by implementing neurons and synapses directly in hardware. ReRAM devices are excellent artificial synapses because their analog, non-volatile, and incremental conductance changes mimic biological synaptic weight updates [13].

Key neuromorphic applications include:

- Spiking neural networks (SNNs): ReRAM devices store and update synaptic weights using spike-timing-dependent plasticity (STDP) rules [33].
- On-chip learning: Multi-level resistance states enable gradual weight adaptation in unsupervised learning.
- Crossbar-based inference engines: Parallel analog current summation performs weighted accumulation inherently in hardware.

Research prototypes such as IBM’s TrueNorth, Intel’s Loihi, and IMEC’s ADRES have explored ReRAM or memristive arrays as dense synaptic fabrics, showing significant energy savings compared to GPU-based neural computation [18].

2.4.5. ReRAM in Edge AI and Embedded Systems

For edge computing and embedded intelligence, ReRAM offers unique benefits:

- Instant-on operation: Non-volatility eliminates boot-up latency.
- Low-standby power: No refresh cycles required.

- In-situ computation: Enables inference or feature extraction within sensor nodes.

ReRAM-based accelerators have been demonstrated for real-time tasks such as image recognition, anomaly detection, and keyword spotting at microwatt power levels [5], [27]. When integrated on FPGA platforms, these prototypes enable hardware-software co-design for domain-specific workloads (e.g., CNN layers, matrix kernels, and recurrent units).

2.4.6. FPGA-Based Emulation of ReRAM PIM Systems

Implementing physical ReRAM arrays for PIM research is costly and time-consuming. Therefore, FPGA emulation has become an essential intermediate step for system exploration. By modeling ReRAM arrays and their peripheral logic in hardware description languages, one can evaluate:

- Data-path throughput and latency for different array configurations.
- AXI4-based communication overhead between ReRAM cores and processors.
- Impact of endurance and switching delay on application performance.

Several works have employed FPGA platforms (Xilinx Zynq, Intel Arria 10, etc.) to prototype neuromorphic and PIM accelerators. These implementations allow real-time execution of workloads such as convolutional neural networks (CNNs) and parameter tuning for endurance-aware scheduling [17], [4].

The ReRAM core developed in this thesis follows this methodology, using parameterizable HDL modules to emulate cell dynamics, timing, and AXI communication, thereby offering a reproducible platform for exploring ReRAM-enabled computation.

2.4.7. Summary

ReRAM's dual role as both memory and computational element unlocks new design paradigms that transcend conventional boundaries between storage and processing. Its crossbar topology enables analog matrix operations for PIM and neuromorphic computing, while its non-volatility and endurance make it ideal for low-power edge systems. FPGA-based emulation bridges the gap between theoretical device models and deployable architectures, allowing rapid evaluation of ReRAM's potential in next-generation computing systems.

2.5. Summary of Literature Review and Research Gap

The preceding sections reviewed the evolution, mechanisms, modeling techniques, and application domains of Resistive Random Access Memory (ReRAM) technology. The review reveals a rapidly maturing field that bridges device physics, compact modeling, and system-level architecture design, yet still exhibits significant gaps between physical research and practical system evaluation.

2.5.1. Summary of Key Findings

1. Non-Volatile Memory Landscape:

The comparison of Flash, MRAM, PCM, FeRAM, and ReRAM established that ReRAM

uniquely combines high scalability, low operating voltage, and analog tunability. Unlike charge-based memories, ReRAM’s resistance-based storage allows for compact, low-power, and potentially 3D-stackable arrays suited to both digital and analog computation [24], [1].

2. Switching Mechanisms:

Resistive switching in ReRAM occurs through ion migration and redox-driven filament formation or interface modulation, enabling reversible transitions between Low- and High-Resistance States. Understanding these physical processes is critical for accurate behavioral modeling and reliability assessment [28], [31].

3. Modeling Approaches:

Modeling spans from physics-based numerical simulations to compact SPICE-level and behavioral hardware-description models.

- Compact models (e.g., TEAM, VTEAM) accurately describe analog I–V characteristics but are computationally expensive and not synthesizable.
- Behavioral models, by contrast, capture key temporal and endurance dynamics and can be implemented in SystemVerilog/VHDL, enabling real-time FPGA emulation [15], [17].

4. Applications in PIM and Neuromorphic Systems:

ReRAM’s analog conductance and crossbar topology enable in-memory matrix–vector multiplication and synaptic weight storage, forming the hardware basis for Processing-in-Memory (PIM) and neuromorphic computing. These paradigms reduce the “memory wall” bottleneck and are being actively explored in accelerators such as ISAAC, PRIME, and NeuroSim [25], [8], [6].

Collectively, these studies confirm ReRAM’s potential as a unifying technology for both non-volatile storage and computation-in-memory, while also highlighting persistent challenges in variability, endurance, and system integration.

2.5.2. Identified Research Gaps

Despite substantial progress, several critical gaps remain in existing research:

1. Limited System-Level Validation Platforms:

Most reported works focus on device- or circuit-level modeling using SPICE or Verilog-A. There is little experimental validation at the architecture or system level using real hardware platforms such as FPGAs. Consequently, evaluating ReRAM performance in realistic data paths, bus interfaces, and embedded processors remains difficult [17].

2. Lack of Parametrizable and Reproducible Frameworks:

Many published ReRAM prototypes are fixed in configuration and not easily adaptable to different cell sizes, delay parameters, or endurance models. Reproducibility and portability across toolchains are often limited. A parameterizable ReRAM core, configurable through HDL parameters or software registers, would allow systematic exploration of different device behaviors under varying workloads.

3. Insufficient Integration with Processor Subsystems:

While ReRAM crossbars have been studied extensively for analog computation, processor-memory interfacing via standard protocols such as AXI4-Lite has received minimal attention. Bridging ReRAM arrays with programmable logic and embedded CPUs is crucial for co-designing hybrid memory–compute architectures [4].

4. Endurance and Timing Behavior in Digital Emulation:

Existing FPGA emulations often omit critical temporal dynamics—such as write delays, endurance degradation, and cell failure flags—resulting in idealized models that fail to capture realistic ReRAM operation.

5. Absence of Open-Source and Educational Platforms:

Most ReRAM test chips or simulators are proprietary or inaccessible. There is a pressing need for open, reproducible FPGA-based emulation frameworks that can serve as both research and teaching tools, accelerating the adoption of resistive memory concepts in system design curricula [26].

2.5.3. Research Direction of This Thesis

Addressing the above gaps, this thesis proposes the development of a parametrizable FPGA-based behavioral model of ReRAM that:

- Implements configurable cell-level parameters such as SET/RESET delay cycles, endurance limit, and state failure behavior;
- Scales into an addressable ReRAM core array with AXI4-Lite interface for seamless integration with ARM processing systems; and
- Provides a reproducible and open platform for evaluating ReRAM-based PIM and neuromorphic architectures.

By bridging device-level models and system-level experimentation, the proposed platform enables quantitative assessment of latency, throughput, and endurance in realistic embedded contexts. This approach supports future explorations of ReRAM as a computational memory and provides a foundation for rapid prototyping of ReRAM-enabled accelerators.

2.5.4. Summary

In conclusion, the literature review underscores the significant progress in understanding ReRAM’s physical mechanisms and potential for next-generation computing. However, it also highlights the lack of flexible, hardware-validated emulation frameworks that connect theoretical models to practical embedded architectures. This thesis directly addresses that need by developing an FPGA-based ReRAM evaluation model, establishing a reproducible link between behavioral modeling and system-level performance characterization.

The design follows a modular and hierarchical structure, ensuring scalability, reusability, and compatibility with future ReRAM-based processing-in-memory (PIM) architectures.



The system implements a behavioral model of a ReRAM array on a Xilinx Zynq-7000 SoC (Zybo Z7-7020) development board. The FPGA’s PL fabric hosts the ReRAM core — consisting of multiple ReRAM cell instances, array management logic, and an AXI4-Lite interface — while the ARM Cortex-A9 processing system (PS) controls the emulation through register-mapped transactions. At a high level, the design enables:

- 20

Figure 5 illustrates the high-level architecture, showing the interaction between hardware and software subsystems.

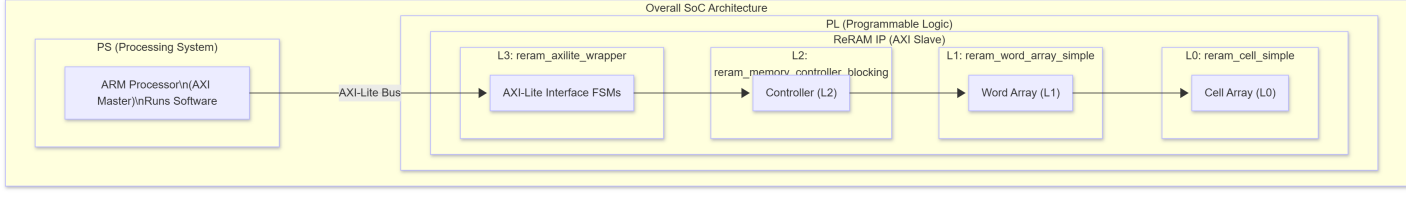


Figure 5: Overall System Architecture

3.2. Hardware Architecture

The hardware subsystem, implemented in the programmable logic (PL) of the Zynq device, forms the foundation of the ReRAM emulation platform.

It is designed using SystemVerilog and integrated within the Vivado IP-Integrator environment.

3.2.1. Hardware Components

The PL subsystem consists of the following primary modules:

1. ReRAM Cell Module (reram-cell)

- Models the functional behavior of a single ReRAM bit-cell.
- Each cell supports two distinct resistance states:
 - Low Resistance State (LRS) $\rightarrow$ Logic '1'
 - High Resistance State (HRS) $\rightarrow$ Logic '0'
- Incorporates:
 - Programmable write delay: to emulate switching latency during SET/RESET.
 - Endurance counter: tracks total write cycles to model device degradation.
 - Failure flag: indicates when the cell exceeds its write limit.
- The cell operates as a finite-state machine (FSM) that transitions between IDLE, SETTING, and RESETTING states based on control signals.

2. ReRAM Core Module (reram-core)

- Aggregates multiple ReRAM cells into a 2D array ($M \times N$).
- Implements address decoding logic to select the target cell for read/write operations.
- Provides row and column selection lines for scalable organization.
- Includes status aggregation logic for reporting busy/failed flags.
- Exposes standardized read and write buses to interface with the AXI bridge.

This modular structure allows easy scaling from small arrays (e.g., 8×8) to larger grids (e.g., 64×64) by simply adjusting the HDL parameters.

3. AXI4-Lite Slave Interface

- Serves as the bridge between the Processing System (PS) and the ReRAM core.
- Implements register-based communication, enabling:
 - Write commands for data input or configuration.
 - Read commands for data and status retrieval.
- The AXI interface ensures memory-mapped access, allowing software control without specialized drivers.

The wrapper implements a full address map to distinguish between memory-access and register-access, as shown in Table 4.

- **Memory Region:** An AXI write or read to this region (e.g., 0x000 - 0x03F) triggers a blocking operation on the ReRAM core. The AXI bus is stalled until the core’s operation (and its associated delay) is complete.
- **Register Region:** An AXI read to this region (e.g., 0x100 - 0x1FF) performs an immediate, non-blocking read of the core’s internal status and monitoring counters. Writes to this region are ignored.

Table 4: ReRAM Memory Control Unit Register Map (Implemented)

Address	Function	Access	Description
0x000-0x03F	MEM_REGION	R/W	16 x 32-bit word memory array. Access is blocking.
0x100	TOTAL_READS	RO	32-bit counter for total read operations.
0x104	TOTAL_WRITES	RO	32-bit counter for total write operations.
0x120-0x15C	WEAR_COUNTERS	RO	Per-word 8-bit endurance counters (packed).

4. Timing and Synchronization Logic

- Synchronizes all cell operations to a global clock (clk).
- Ensures proper sequencing of SET/RESET events and read operations.
- Maintains deterministic timing suitable for cycle-accurate emulation.

5. Debug and Monitoring Unit

- Captures internal states (FSM transitions, endurance counters, error flags).
- Accessible via AXI registers for visualization and post-processing.
- Used for validating timing and reliability behavior under repeated write cycles.

3.2.2. FPGA Implementation Strategy and Resource Profile

A key architectural decision is the implementation of the ReRAM array **entirely in programmable logic** rather than utilizing on-chip Block RAM (BRAM). This approach, while resource-intensive, is essential for a true *behavioral* emulation. Each `reram_cell_simple` instance is synthesized into its

own set of Flip-Flops (FFs) and Look-Up Tables (LUTs) to manage its state, endurance counter, and delay-based FSM.

As a result, the design’s resource footprint scales linearly with the number of emulated cells, as confirmed in the experimental analysis (Section 6.5). The final implementation of the 16x32 (512-bit) core demonstrates this, consuming a significant portion of the Zybo Z7-7020’s logic fabric (**34,545 LUTs and 26,062 FFs**) but **zero BRAM** (Table 6.1). This confirms the design’s success as a logic-based behavioral model, where the cost is intentionally traded for the ability to accurately model cycle-by-cycle timing and wear-out phenomena.

3.2.3. System Interconnect

The interconnection between modules follows the AXI4-Lite standard, ensuring compatibility with the PS software environment.

The bus handles:

- Control and configuration writes from the ARM processor.
- Readback of ReRAM states and cell data.
- Interrupt or polling-based signaling for status updates.

A global clock and reset domain synchronizes all hardware elements, ensuring reproducible and deterministic operation.

3.3. Software Architecture

The software subsystem runs on the dual-core ARM Cortex-A9 within the Zynq Processing System (PS). It provides high-level control, data logging, and evaluation interfaces for interacting with the ReRAM hardware core.

3.3.1. Software Components

1. Device Driver Layer

- Implements low-level AXI4-Lite read/write access functions using Xilinx bare-metal SDK APIs or PetaLinux drivers.
- Provides abstractions such as:

Listing 2: High-Level Functions for ReRAM Access

```
1 reram_write(address, data);  
2 data = reram_read(address);  
3 reram_check_status();
```

- Ensures proper handshake synchronization and timing between PS and PL.

2. Validation Application Layer

- Provides a bare-metal C application for running validation tests:
 - Functional tests: Verify write/read correctness.

- Endurance tests: Continuously toggle cells to emulate wear-out and read status registers to monitor wear.
- Timing tests: Measure latency between SET and RESET operations.
- Logs results to a host PC via UART.

3. Data Logging and Visualization

- Uses UART interfaces to send data to a host PC.
- Enables post-processing in Python or MATLAB to visualize endurance degradation or timing distributions.

3.3.2. Software–Hardware Communication Flow

The communication flow between software and hardware is summarized as follows:

1. Initialization:

- The PS configures the AXI interface, resets the ReRAM core, and loads parameters such as delay constants and endurance limits.

2. Write Cycle (SET/RESET):

- The PS writes to the CONTROL and ADDRESS registers, followed by the WRITE-DATA register.
- The AXI bridge triggers the internal ReRAM FSM, initiating the state transition.

3. Polling and Completion:

- The PS polls the STATUS register until the busy flag clears.
- Once complete, it reads back the updated cell state through READ-DATA.

4. Monitoring and Logging:

- Results are timestamped and stored for analysis.
- Optional UART-based logging provides a real-time console output of cell states and endurance counters.

5. Error Handling:

- If the endurance limit is reached, the ReRAM core raises the failure flag, which the PS interprets for failure statistics.

3.3.3. Development Environment

- Hardware Design Tools:
 - Xilinx Vivado Design Suite 2018.1 (HDL synthesis, IP integration)
 - Vivado SDK / Vitis 2018.1 (bare-metal firmware development)
- Simulation Tools:

- ModelSim / Vivado Simulator for behavioral verification.
- GTKWave for waveform analysis.
- Software Stack:
 - C or C++ application for control logic.
 - Optional Python interface for higher-level testing via UART.

3.4. Integration and System Workflow

1. The ReRAM behavioral model is described in SystemVerilog.
2. The HDL design is synthesized and exported as an AXI4-Lite IP block.
3. The block is integrated with the Zynq PS in Vivado’s block design.
4. The bitstream and hardware definition are exported to Vitis, where the C-based control software is compiled.
5. The application executes on the ARM processor, performing test operations on the ReRAM array. This workflow creates a closed-loop hardware–software validation platform, enabling iterative tuning of parameters and real-time performance measurement.

3.4.1. Advantages of the Architecture

- Reproducibility: Fully deterministic and parameter-controlled emulation.
- Flexibility: Adjustable ReRAM cell parameters allow diverse test conditions.
- Scalability: Suitable for extending to multi-array or PIM configurations.
- Compatibility: AXI4-Lite ensures seamless integration with standard SoC environments.
- Real-Time Insight: Hardware execution speed allows empirical evaluation of delay, endurance, and fault behavior.

3.4.2. Summary

The system architecture effectively bridges the gap between device-level ReRAM models and system-level implementation. The hardware subsystem provides a cycle-accurate emulation of ReRAM dynamics, while the software subsystem offers flexible control and data analysis. This co-designed platform forms the foundation for subsequent chapters, which focus on implementation details, synthesis results, and performance evaluation.

4. Implementation

4.1. Introduction

This chapter details the design and implementation of a hierarchical, synthesizable memory subsystem model based on Resistive Random-Access Memory (ReRAM) technology. The primary objective of this design is to provide a realistic, behavior-driven hardware module that accurately captures the defining characteristics of ReRAM, namely:

1. **Non-Instantaneous Operations:** Unlike SRAM, ReRAM cells require a finite, non-trivial amount of time (on the order of nanoseconds to microseconds) to perform read, set (write '1'), and reset (write '0') operations.
2. **Write Endurance:** ReRAM cells have a finite lifespan and can only withstand a limited number of write cycles before their physical properties degrade, leading to failure.

The implementation is structured in a bottom-up, four-layer hierarchy, promoting modularity, parameterization, and scalability. This hierarchy allows for the modeling of physical cell behaviors at the lowest level, which are then abstracted up to a standard, system-level AXI-Lite bus interface at the highest level.

The four layers of the design as shown in Figure 6 are:

- **L0: ReRAM Cell (`reram_cell_simple`):** The fundamental building block, modeling a single ReRAM bit.
- **L1: ReRAM Word Array (`reram_word_array_simple`):** A parallel aggregation of cells to form a single memory word.
- **L2: Memory Controller (`reram_memory_controller_blocking`):** An addressable array of words, presenting a simple SRAM-like blocking interface.
- **L3: AXI-Lite Wrapper (`reram_axilite_wrapper`):** The top-level module, providing a standard AXI-Lite interface to the memory controller.

This chapter will analyze each module in order, from the bottom-level cell to the top-level system interface.

4.2. L0: The ReRAM Cell (`reram_cell_simple`)

The `reram_cell_simple` module shown in Figure 7 is the atomic unit of the memory system. It does not represent a simple register but rather a stateful model of a single physical ReRAM cell.

Note: The complete RTL for this module can be found in Listing 4 in Appendix A on page 54.

4.2.1. Purpose:

To emulate the time-domain behavior and physical limitations of one ReRAM bit.

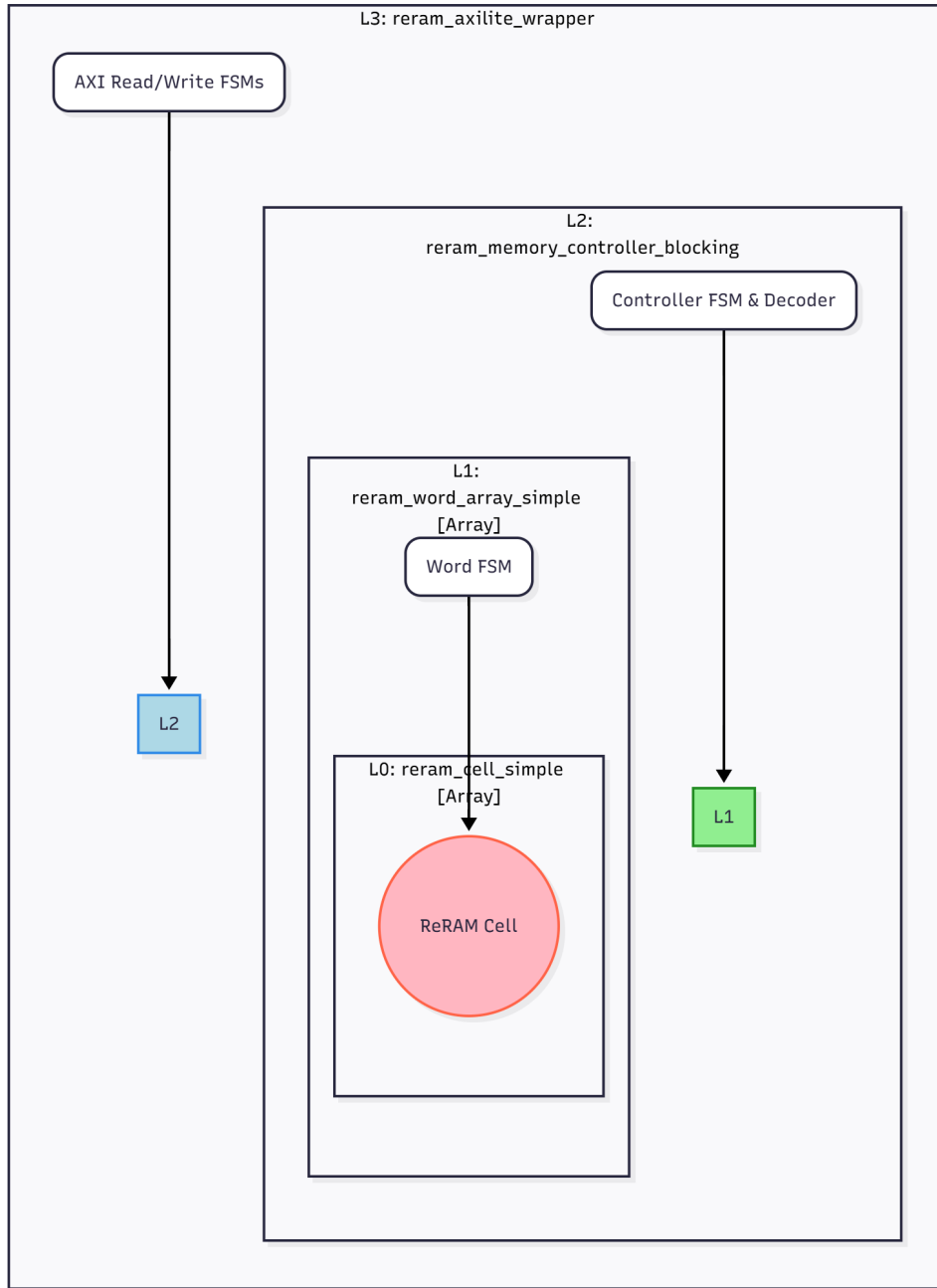
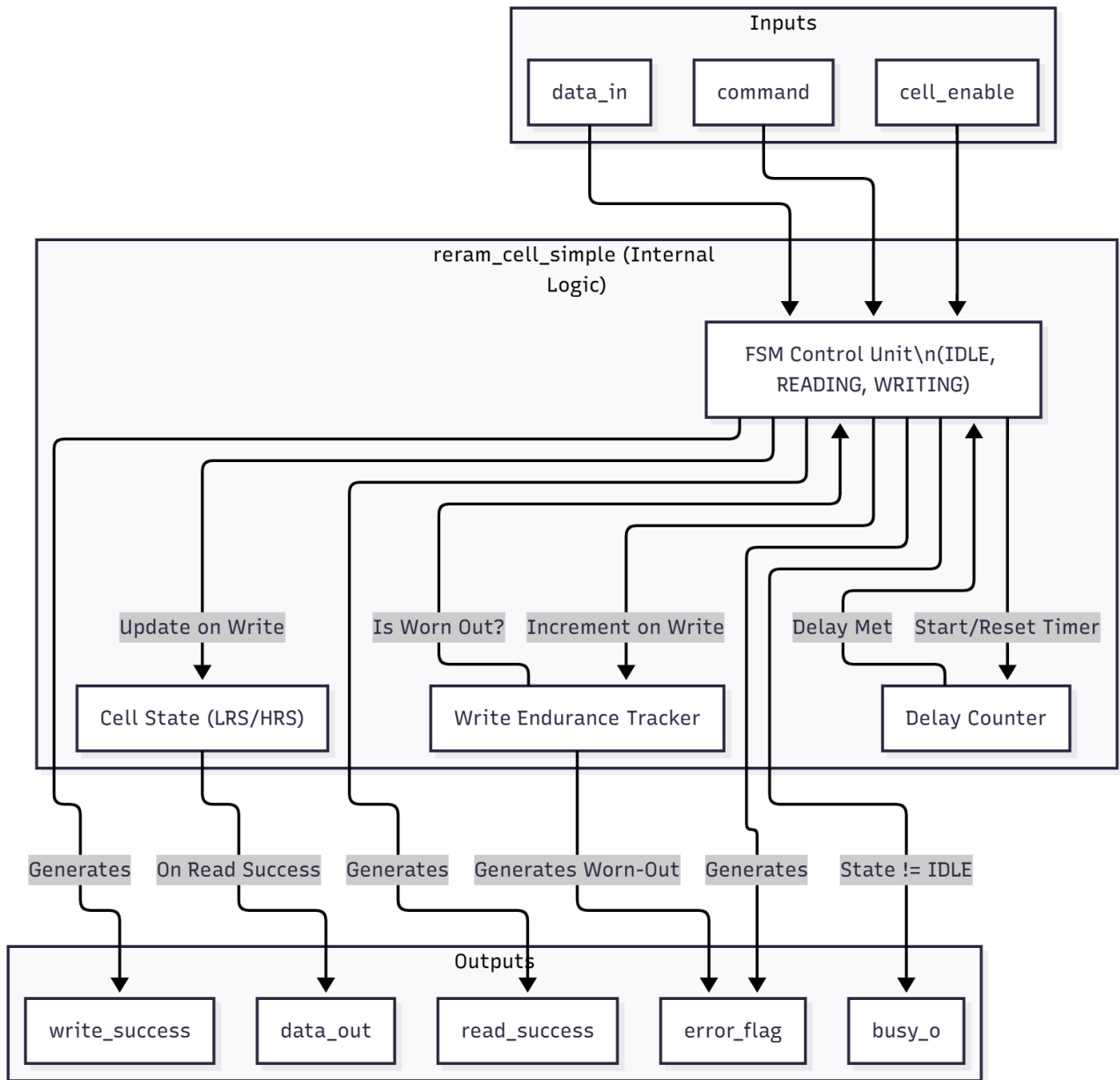


Figure 6: Design Hierarchy

4.2.2. Key Features:

- **Parameterization:** Configurable delays for `READ_DELAY_CYCLES`, `SET_DELAY_CYCLES`, and `RESET_DELAY_CYCLES` allow the module to be tailored to different ReRAM technology characteristics.
- **Endurance Tracking:** A `write_counter` tracks the number of SET/RESET operations. When this count exceeds the `MAX_WRITE_CYCLES` parameter, the cell enters a "worn-out" error state and will no longer accept write commands.
- **Binary State:** The `internal_cell_state` register holds the cell's logical value, representing the Low-Resistance State (LRS) as '1' and the High-Resistance State (HRS) as '0'.

Figure 7: Logical dataflow diagram for the L0 ReRAM Cell (`reram_cell_simple`).

- **Error Reporting:** The cell reports `ERR_BUSY` if a new command is received while an operation is in progress, and `ERR_WORN_OUT` if the endurance limit is breached.

4.2.3. State Machine Implementation:

The core of the cell is a 4-state Finite State Machine (FSM) that manages the operation latencies.

- **IDLE:** The cell is waiting for a command. Upon receiving an `active_read_cmd` or `active_write_cmd`, it latches the appropriate target delay (`READ_DELAY_CYCLES`, etc.) and transitions.
- **READING:** The FSM remains in this state while the `delay_counter` increments to the `target_delay`. During this time, the cell is "busy." Once the delay is met, it asserts `operation_complete` and returns to **IDLE**.
- **WRITING_SET / WRITING_RESET:** Similar to **READING**, these states enforce the write latencies. Upon completion, they update the `internal_cell_state` to '1' (LRS) or '0' (HRS) respectively, increment the `write_counter`, and return to **IDLE**.

This FSM is crucial as it enforces the *blocking* nature of ReRAM. A `read_success` or `write_success` pulse is only generated *after* the specified delay has elapsed, accurately modeling the cell's physical response time.

4.3. L1: The ReRAM Word Array (`reram_word_array_simple`)

4.3.1. Purpose:

To group `WORD_WIDTH` (e.g., 32) individual `reram_cell_simple` modules shown in Figure 8 into a single, cohesive memory word that can be read or written in parallel.

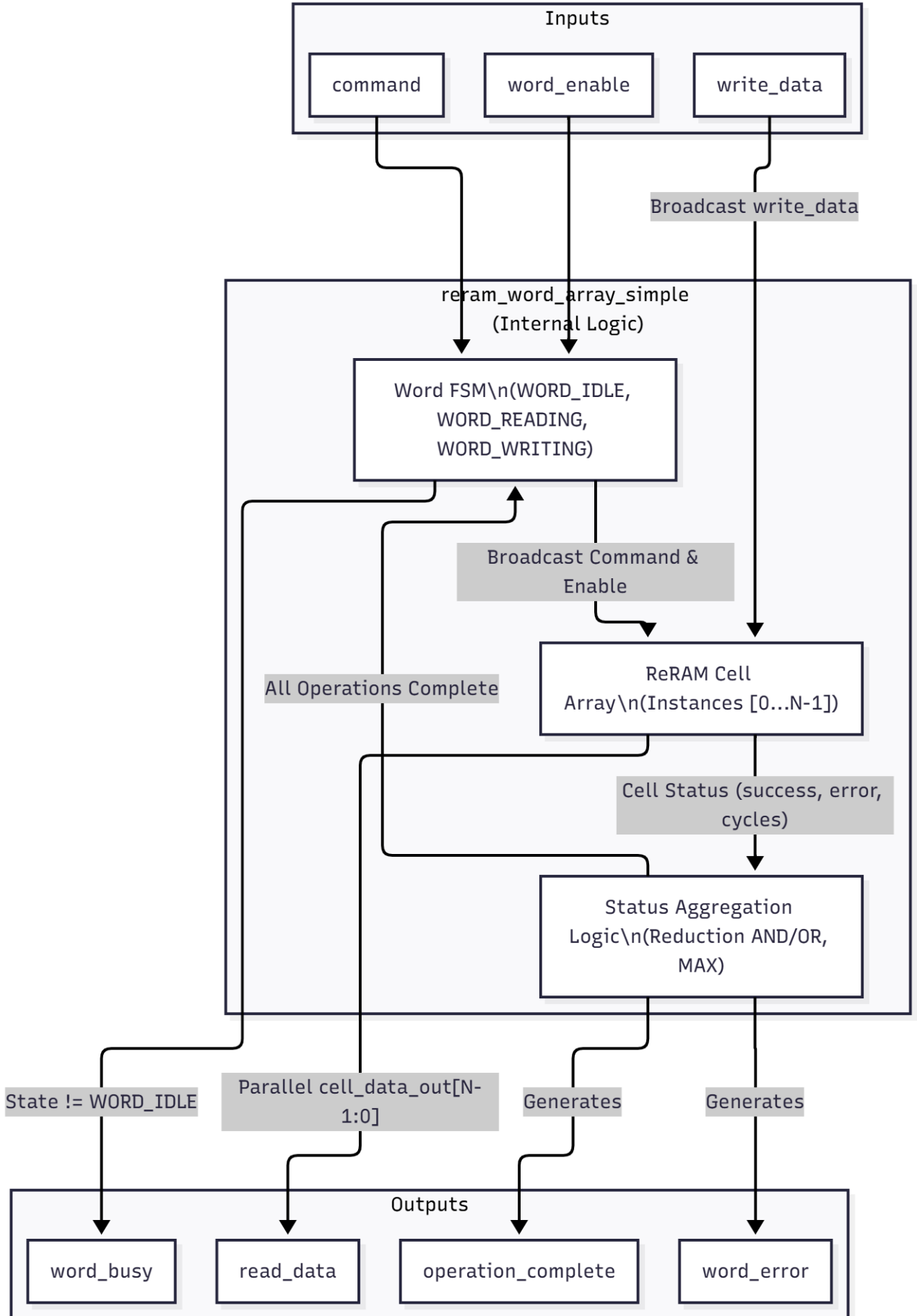
Note: The complete RTL for this module can be found in Listing 5 in Appendix A on page 59.

4.3.2. Architecture:

This is a structural module. A `generate` loop instantiates an array of `reram_cell_simple` modules. The core logic of this module is focused on **command broadcasting** and **status aggregation**.

4.3.3. Functional Analysis:

- **Command Broadcasting:** A simplified word-level FSM (`WORD_IDLE`, `WORD_READING`, `WORD_WRITING`) is used to coordinate the cells. When a `word_enable` is received in the `WORD_IDLE` state:
 1. The FSM transitions to `WORD_READING` or `WORD_WRITING`.
 2. On this transition, it emits a **one-cycle command pulse** (`cell_command_reg`) and asserts `cell_enables_reg` for all cells in the word.
 3. All `WORD_WIDTH` cells receive this command simultaneously and enter their respective busy states (e.g., `READING`).

Figure 8: Logical dataflow diagram for the L1 ReRAM Word Array (`reram_word_array_simple`).

- **Synchronization and Completion:** The word array's primary challenge is synchronization. The `word_enable` and `command` inputs may be de-asserted, but the word remains *busy* until all constituent cells have finished their internal delays.
 - This is managed by the `all_read_complete` and `all_write_complete` logic. These signals perform a reduction-AND (&) across the `cell_read_success` (or `write_success`) and `cell_error_flag` vectors.
 - The operation is only considered complete when *every cell* has either reported success or an error. This correctly models the worst-case latency of the parallel operation.
- **Status Aggregation:** The module aggregates status from all cells. `word_error` is a reduction-OR (|) of all `cell_error_flags`. `total_write_cycles` is calculated by finding the *maximum* value among all `cell_write_cycles`, representing the most-worn cell in the word.

4.4. L2: The Memory Controller (`reram_memory_controller_blocking`)

4.4.1. Purpose:

Shown below in Figure 9 it is used to abstract an array of `MEMORY_DEPTH` word arrays into a conventional, SRAM-like memory. It provides a simple blocking interface (`read_i`, `write_i`, `addr_i`) and hides the complexity of the underlying word-level operations.

Note: The complete RTL for this module can be found in Listing 6 in Appendix A on page 60.

4.4.2. Architecture:

This module instantiates an array of `reram_word_array_simple` modules, one for each address in the memory space.

4.4.3. Functional Analysis:

- **Interface:** This module's interface is transactional. The user asserts `read_i` or `write_i` with a valid `addr_i` to begin an operation. The controller asserts `busy_o` until the operation is finished, at which point it asserts `done_o` for one cycle.
- **Address Decoding:** The `addr_i` input is latched into `current_addr` when an operation begins. This `current_addr` is used to generate a one-hot `word_enables` signal, activating *only* the single `reram_word_array_simple` instance being addressed.
- **Blocking FSM:** A simple 4-state FSM (IDLE, READING, WRITING, ERROR_STATE) orchestrates the transaction.
 1. **IDLE:** Waits for `read_i` or `write_i`. Upon assertion, it latches `addr_i` and `data_i` and transitions to the active state (e.g., READING).
 2. **READING / WRITING:** In this state, the controller issues the one-cycle command (`word_command_reg`) to the selected word array (via the `word_enables` logic). The FSM then *waits*. It monitors the `word_operation_complete` signal from *only the addressed word*.

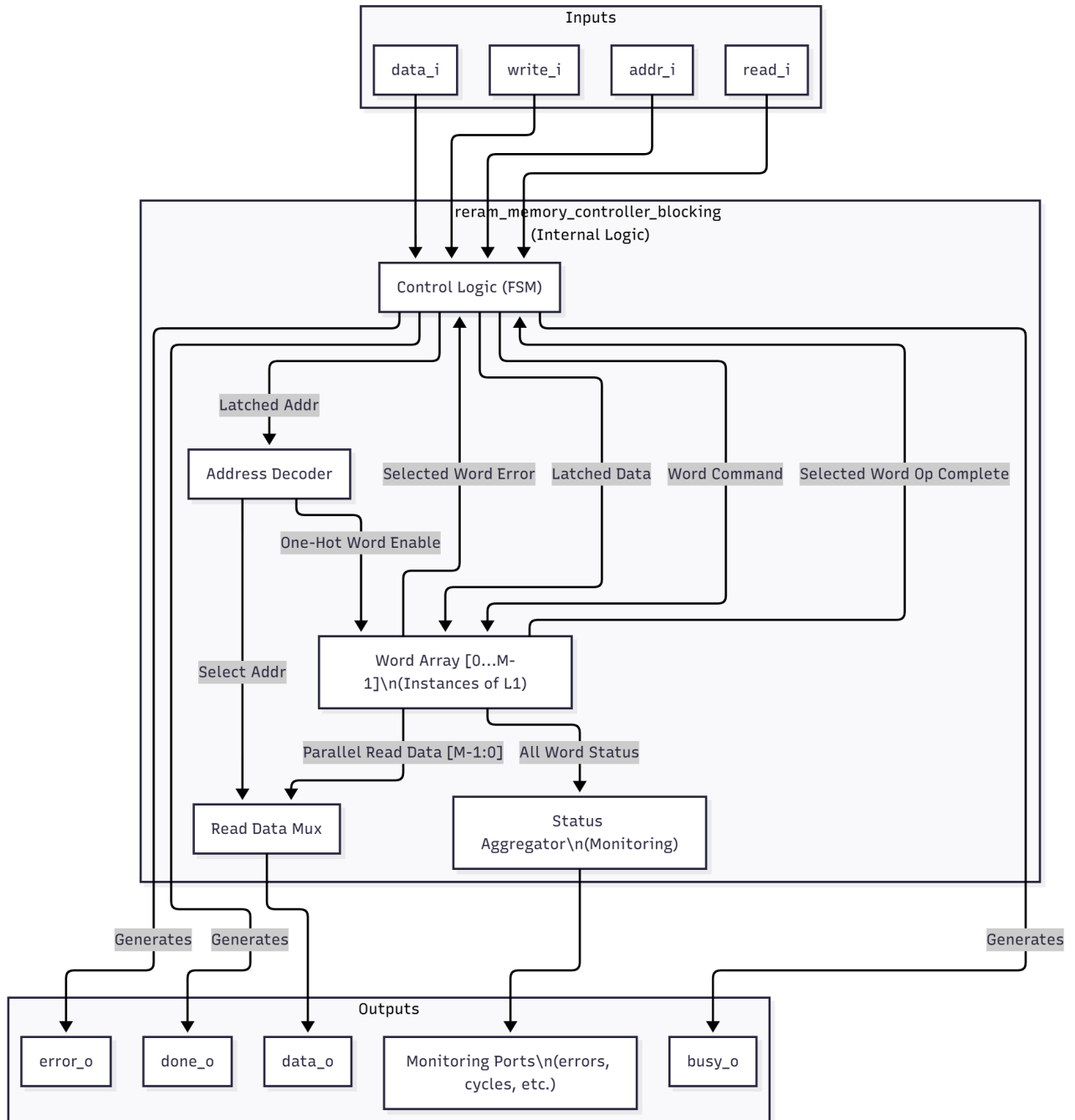


Figure 9: Logical dataflow diagram for the L2 Memory Controller (`reram_memory_controller_blocking`).

3. Once `word_operation_complete[current_addr]` is asserted, the FSM transitions back to IDLE.

- **Operation:** The controller's `busy_o` signal is tied directly to its FSM state. Therefore, the controller remains "busy" for the *entire duration* of the underlying ReRAM operation (e.g., `SET_DELAY_CYCLES`). This "blocking" behavior is the key abstraction this module provides.

4.5. L3: The AXI-Lite Wrapper (`reram_axilite_wrapper`)

4.5.1. Purpose:

Shown below in Figure 10 it is used to integrate the proprietary, blocking interface of the `reram_memory_controller` core into a standard, bus-based system using the AXI-Lite protocol.

Note: The complete RTL for this module can be found in Listing 7 in Appendix A on page 63.

4.5.2. Address Mapping:

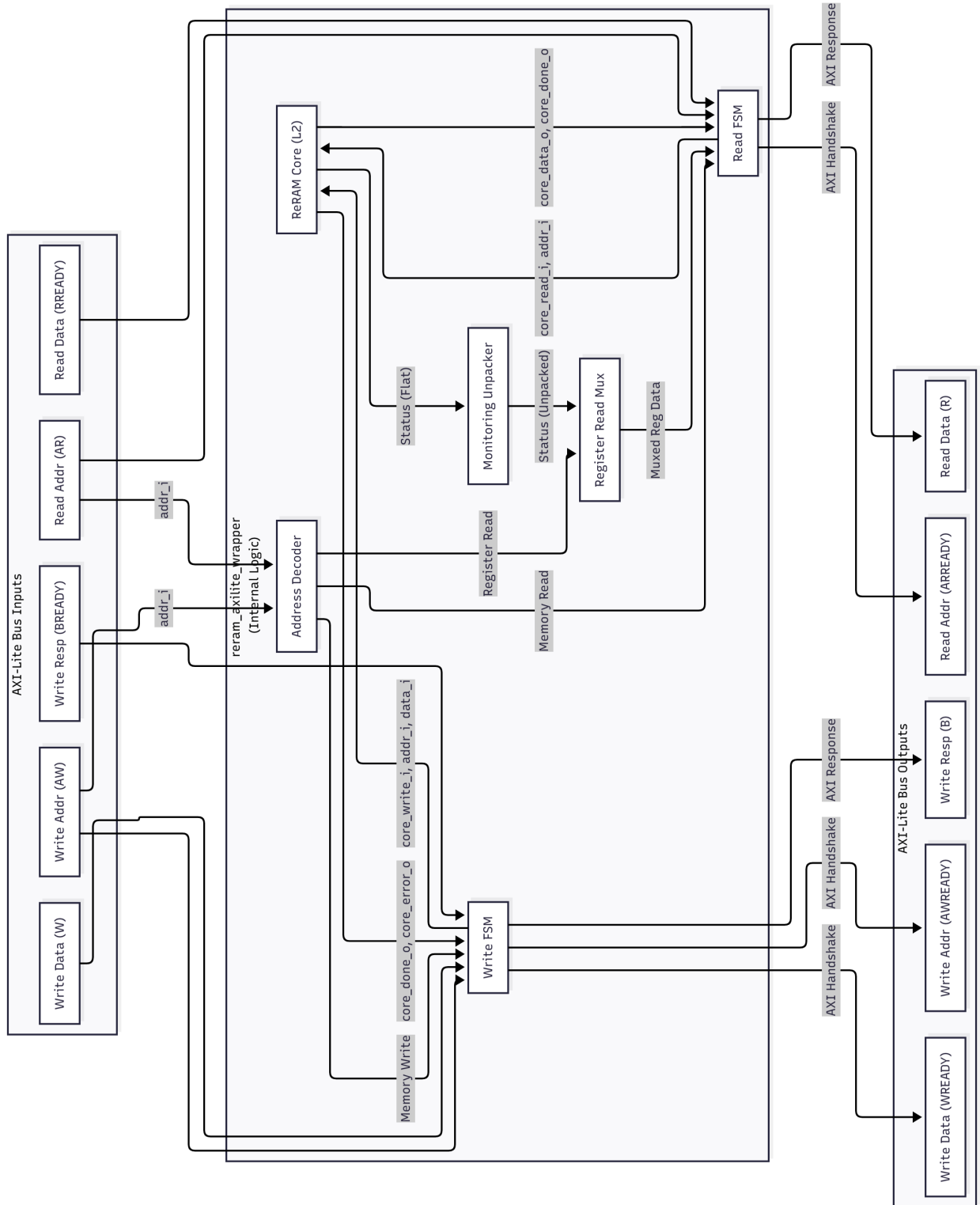
The wrapper implements an address map to distinguish between accesses to the ReRAM memory and accesses to internal Control/Status Registers (CSRs).

- **Memory Region (e.g., 0x000 - 0x03F):** Accesses to this region trigger blocking read/write operations on the ReRAM core.
- **Register Region (e.g., 0x100 - 0x23F):** Accesses to this region perform non-blocking (immediate) reads of the core's status and monitoring outputs (e.g., `total_reads`, `total_writes`, `word_write_cycles`).

4.5.3. Transaction Translation:

The core of the wrapper consists of two AXI FSMs (one for read, one for write) that perform "transaction translation."

- **AXI Write to Memory:**
 1. The FSM waits for both `S_AXI_AWVALID` and `S_AXI_WVALID` (`WR_IDLE` state).
 2. It accepts the transaction, latches the address/data, and asserts `core_write_i` for *one cycle*.
 3. It transitions to `WR_WAIT_CORE` and **stalls the AXI write response**. `S_AXI_BVALID` is held low.
 4. When `core_done_o` is asserted, the FSM transitions to `WR_RESP`, asserts `S_AXI_BVALID`, and completes the AXI transaction.
- **AXI Read from Memory:**
 1. The FSM accepts `S_AXI_ARVALID`, latches the address, and pulses `core_read_i`.

Figure 10: Logical dataflow diagram for the L3 AXI-Lite Wrapper (`reram_axilite_wrapper`).

2. It transitions to RD_WAIT_CORE and **stalls the AXI read data channel**. S_AXI_RVALID is held low.
 3. When core_done_o is asserted, it latches the core_data_o.
 4. It transitions to RD_RESP, asserts S_AXI_RVALID, and presents the latched data.
- **AXI Read from Registers:** If an AXI read address maps to the CSR region, the read FSM bypasses the RD_WAIT_CORE state. It combinationally selects the register data and asserts S_AXI_RVALID on the next cycle for an immediate, non-blocking read.

4.6. Conclusion

This hierarchical design successfully models a ReRAM memory subsystem for use in a standard hardware design flow. By encapsulating the core ReRAM behaviors of latency and endurance within the `reram_cell_simple` module, the system can realistically simulate the performance and wear-out of the memory. The subsequent layers of abstraction—`reram_word_array_simple` and `reram_memory_controller_blocking`—effectively scale this behavior from a single bit to a full memory array, while the `reram_axilite_wrapper` provides a standard, plug-and-play interface for system-on-chip (SoC) integration. The design is parameterizable, synthesizable, and provides extensive monitoring capabilities critical for evaluating ReRAM-based architectures.

5. Experimental Setup and Implementation

5.1. Introduction

This section details the verification, implementation, and analysis of the ReRAM memory subsystem. The primary objectives are:

1. **Functional Verification:** To rigorously test the SystemVerilog modules using a hierarchical testbench environment to ensure logical correctness and adherence to the specified behaviors (e.g., operation latency, endurance, error reporting).
2. **Hardware Implementation:** To synthesize and implement the entire design, targeting a specific Xilinx FPGA, to demonstrate its viability as a hardware IP core.
3. **Resource and Power Analysis:** To quantify the hardware footprint of the design by analyzing its utilization of key FPGA resources (LUTs, FFs, BRAMs) and estimating its power consumption.
4. **System Integration:** To outline the software-level interaction with the core via the Xilinx Software Development Kit (SDK) or Vitis, demonstrating a complete hardware-software co-design flow.

The methodology follows the standard Xilinx Vivado design flow, from RTL simulation to bitstream generation and software application development.

5.2. Design and Implementation Methodology

The implementation of the ReRAM memory subsystem was carried out using the Xilinx Vivado Design Suite.

- **Design Suite:** Xilinx Vivado 2018.1 (This is specified based on the Vivado 2018.1 compatibility fixes in the `reram_axilite_wrapper` code).
- **Target Platform:** Zybo Z7-20 (xc7z020clg400-1)
- **Simulation Tool:** Vivado Simulator

5.2.1. Vivado Project Flow

The ReRAM IP was integrated into a larger system-on-chip (SoC) design using the Vivado IP Integrator, following these steps:

1. **RTL Project Creation:** The SystemVerilog files (`reram_cell_simple.sv`, `reram_word_array_simple.sv`, `reram_memory_controller_blocking.sv`, and `reram_axilite_wrapper.sv`) were imported into a new Vivado project.
2. **IP Packaging:** The top-level module, `reram_axilite_wrapper`, was packaged as a custom AXI-Lite slave IP using the "Create and Package New IP" wizard.

3. **Block Design Creation:** A new block design was created using the IP Integrator.
4. **PS Integration:** The Zynq-7000 Processing System (PS) was added to the block design.
5. **IP Addition:** The newly packaged ReRAM IP (`reram_axilite_wrapper`) was added from the IP catalog to the block design.
6. **Connection Automation:** Vivado's "Run Connection Automation" tool was used to automatically connect the AXI-Lite interface of the ReRAM IP to the `M_AXI_GP0` port of the Zynq PS.
7. **Address Assignment:** An AXI address was assigned to the ReRAM IP (e.g., `0x43C00000`) via the Address Editor.
8. **Bitstream Generation:** The design was validated, an HDL wrapper was generated, and the design was synthesized, implemented (Place & Route), and a bitstream was generated.

The final system block design, as shown in the Vivado IP Integrator, is provided in Figure 11.

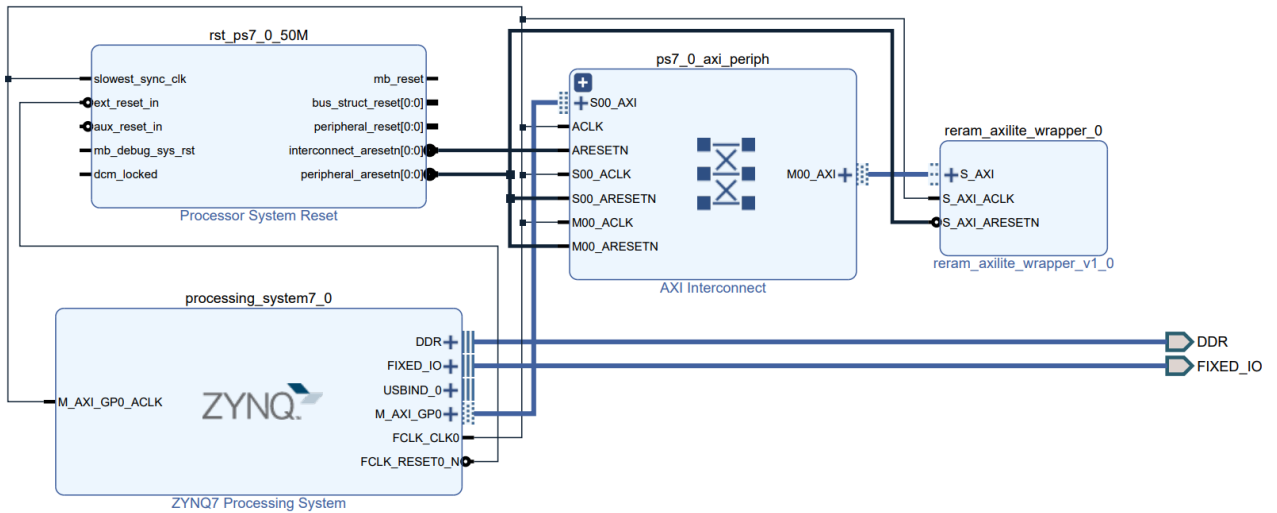


Figure 11: System block design showing the Zynq PS connected to the ReRAM AXI-Lite IP.

5.3. Functional Verification (Simulation)

Functional verification was performed using a hierarchical, bottom-up approach. Testbenches were created for each layer of the design (Cell, Word, Controller, and AXI Wrapper) to isolate and validate specific functionalities before integrating them into the next level. All testbenches are written in SystemVerilog and use directed, task-based test sequences.

5.3.1. L0: Cell-Level Testbench (`tb_reram_cell_simple`)

This testbench performs a "unit test" on the most fundamental component, the `reram_cell_simple` module.

Testbench Design and Operation

This testbench directly manipulates the command inputs (command, cell-enable, data-in) to simulate individual operations. It uses SystemVerilog ‘task’s to abstract these operations into clear, readable commands like `write_set()`, `write_reset()`, and `read_cell()`.

The test sequence is divided into phases:

- **Phase 1 (Basic Operations):** Verifies that a `write_set` followed by a `verify_value` correctly reads a ‘1’, and a `write_reset` correctly reads a ‘0’.
- **Phase 3 (Timing):** The `test_timing` task measures the cycles between issuing a command and receiving the corresponding success pulse (`read_success` or `write_success`), verifying it matches the `READ_DELAY` and `SET/RESET_DELAY` parameters.
- **Phase 4 (Error Conditions):** Checks that the cell correctly ignores commands when `cell_enable` is low.
- **Phase 6 (Endurance Tracking):** Performs 20 write operations and checks that the `write_cycles` output counter correctly increments from its initial value to (initial + 20).
- **Phase 7 (Endurance Limit):** This is a destructive test that writes to the cell repeatedly in a `while` loop until the `write_cycles` counter reaches the `MAX_WRITES` limit. It then verifies that the `error_flag` asserts with the correct `ERR_WORN_OUT` code, and that subsequent write attempts are ignored.

Note: The complete SystemVerilog code for this testbench can be found in Listing 8 in Appendix B on page 66.

Simulation Results and Waveform Analysis

- **Console Output:** The console log provides a real-time narrative of the test. Key observations are the ‘[PASS] Data verification passed’ messages and the ‘Write count progression’ log in Phase 6. The final summary, ‘*** ALL CELL TESTS PASSED! ***’, confirms all checks were successful.
- **Waveform Analysis:** Analysis of the L0 cell waveform (Figure 12) confirms the correct timing and stateful behavior. In the figure, a `CMD_SET` operation is initiated (1). It is observed that the `write_success` signal (2) pulses exactly 10 clock cycles after the command, precisely matching the `SET_DELAY` parameter. This observation validates the correct operation of the internal delay counter and FSM. Concurrently, the `cell_state` output (3) transitions to ‘1’ only after the `write_success` pulse, confirming the state is updated upon completion and not prematurely.

5.3.2. L1: Word-Array-Level Testbench (tb_reram_word_array_simple)

This testbench performs an integration test on the `reram_word_array_simple` module, which consists of 32 parallel L0 cells.

Testbench Design and Operation

This testbench, interacts with the DUT's word-level interface ('command', 'word_enable', 'write_data'). It does not see the individual cells, only the aggregated 32-bit 'read_data' and status signals.

The test sequence validates the DUT's two main responsibilities: command broadcasting and status aggregation.

- **Phase 1-3 (Bit Integrity):** These phases test command broadcasting. They write various 32-bit patterns (e.g., '0xDEADBEEF', "All Zeros," "All Ones," "Walking 1s") and verify that the *exact* 32-bit pattern is read back. A "Walking 1s" test, for example, confirms that writing to one bit (e.g., bit 7) does not accidentally affect adjacent bits (e.g., bit 6 or 8).
- **Phase 4 (Timing):** This test verifies that the 'operation_complete' pulse respects the delay of the underlying cells.
- **Phase 6 (Endurance Tracking):** This tests status aggregation. It performs several writes and verifies that the 8-bit 'total_write_cycles' output correctly increments.
- **Phase 9 (Endurance Limit):** This is a destructive test that writes to the word array repeatedly until its 'total_write_cycles' counter reaches the MAX_ENDURANCE_CYCLES limit. It then verifies that the 'word_error' and 'cell_error_flags' outputs assert, and that all subsequent write and read operations fail as expected.

Note: The complete SystemVerilog code for this testbench can be found in Listing 9 in Appendix B on page 68.

Console and Waveform Analysis

- **Console Output:** The log shows word-level operations ("Writing word: 0x..."). The '[HEALTH]' check, which shows the "Write utilization" percentage, is key to monitoring the test's progress. The final summary, "*** ALL WORD ARRAY TESTS PASSED! ***", confirms success.
- **Waveform Analysis:** The most important signal to observe is `word_busy`. After a 1-cycle command pulse, 'word_busy' must assert and remain high for the *entire* duration of the operation (e.g., 10 cycles). The `operation_complete` signal should pulse for one cycle at the exact moment 'word_busy' de-asserts.

5.3.3. L2: Controller-Level Testbench (tb_reram_memory_controller_blocking)

This testbench validates the `reram_memory_controller_blocking` module, which adds an addressable memory array on top of the L1 words.

Testbench Design and Operation

This testbench emulates a simple CPU or state machine by interacting with the SRAM-like interface (read-i, write-i, addr-i). Its tasks, `write_memory()` and `read_memory()`, are designed to test the module's key feature: the blocking handshake. A task asserts 'write-i' for one cycle, then immediately waits for the 'done-o' signal, which may take many cycles.

Key test phases include:

- **Phase 1-4 (Address/Data Tests):** These tests write different data patterns to *different addresses* (e.g., 0x0, 0x1, 0xF) and then read them back. This verifies that the address decoder is working and that writing to one address (e.g., 0x0) does not corrupt the data in another (e.g., 0x1).
- **Phase 6 (Endurance Counter Testing):** This is the most important test. It records the initial write cycle count for address 0x0 and 0x1. It then performs 10 writes to address 0x0 and 5 writes to address 0x1. Finally, it verifies that the ‘word_write_cycles’ counter for address 0x0 incremented by exactly 10, and the counter for 0x1 incremented by 5. This proves that the per-address endurance tracking is independent and correct. It also verifies that ‘read’ operations do *not* increment the write counter.
- **Phase 7 (Wear-Out):** This test uses the `wear_out_address` task to write to a single address (e.g., 0xF) 200 times, intentionally destroying it. It then verifies that subsequent writes and reads to 0xF return an error, while writes and reads to other addresses (e.g., 0x0) still work perfectly.

Note: The complete SystemVerilog code for this testbench can be found in Listing 10 in Appendix B on page 69.

Simulation Results and Waveform Analysis

- **Console Output:** The log shows operations targeted at specific addresses ("Writing... to address 0x0004"). The Phase 6 log is crucial, showing "Address 0x0 write counter correctly incremented by 10" and "Address 0x1 write counter correctly incremented by 5", which verifies the independence of the per-address endurance counters.
- **Waveform Analysis:** The waveform for the L2 controller validates the simple blocking handshake. The testbench asserts `write_i` for a single cycle (1) to initiate a write to a specific `addr_i`. In response, the controller immediately asserts `busy_o` (2). This `busy_o` signal remains high for the full 10-cycle write delay. When the internal operation completes, the controller asserts `done_o` for one cycle (3), signaling the end of the transaction. The `busy_o` signal de-asserts concurrently. This observation proves the controller correctly abstracts the ReRAM's latency into a simple, blocking SRAM-like interface.

5.3.4. L3: AXI-Wrapper-Level Testbench (`tb_reram_axilite_wrapper`)

This is the top-level simulation, verifying the final IP core. This testbench emulates an AXI-Lite Master, such as the Zynq Processing System (PS).

Testbench Design and Operation

The testbench, validates the translation from AXI-Lite bus transactions to the simple blocking interface of the L2 controller. It uses complex tasks, `axi_write()` and `axi_read()`, which manage the full multi-channel AXI handshake.

The test sequence is critical as it must validate the two separate address regions:

- **Phase 1 (Memory Region):** This phase performs AXI writes and reads to the memory region (e.g., ‘0x000’ to ‘0x03C’). It writes ‘0xDEADBEEF’ to address ‘0x000’, ‘0xCAFEBAFE’ to ‘0x004’, and then reads them back to verify data integrity.
- **Phase 2 (Register Region):** This phase tests the memory-mapped status registers. After the 3 writes and 3 reads in Phase 1, it performs an AXI read from address ‘0x100’ (ADDR_TOTAL_READS) and verifies the returned value is 3. It then reads ‘0x104’ (ADDR_TOTAL_WRITES) and also verifies the value is 3.
- **Phase 3 (Endurance Register):** This phase performs 5 more writes to a new address (‘0x008’) and then reads the packed endurance counter register at ‘0x120’. It verifies the returned 32-bit value correctly contains the individual 8-bit counters (e.g., ‘8’d5’ for word 2).
- **Phase 4 (Read-Only Test):** This phase intentionally attempts an AXI write to the read-only ‘ADDR_TOTAL_READS’ register (‘0x100’). It then reads the register back to verify that the value was *not* changed, confirming the read-only logic is working.

Note: The complete SystemVerilog code for this testbench can be found in Listing 11 in Appendix B on page 71.

Simulation Results and Waveform Analysis

- **Console Output:** The log shows AXI transactions (“AXI Write: ... to Addr 0x...”). The ‘[PASS]’ messages in Phase 2 and 3 are the most important, as they confirm that the status registers (e.g., ADDR_TOTAL_READS) are being correctly read over the AXI bus.
- **Waveform Analysis:** The L3 AXI-wrapper waveform (Figure 13) validates the critical translation from AXI transactions to the core’s blocking interface. Two key behaviors are shown:
 1. **Blocking Memory Write:** The testbench initiates a write by asserting S_AXI_AWVALID and S_AXI_WVALID (1). The DUT accepts the transaction but immediately *stalls the AXI bus* by holding the write response signal S_AXI_BVALID low (2). This stall persists until the internal L2 controller signals completion via core_done_o (3). Only after this internal handshake does the wrapper assert S_AXI_BVALID (4), completing the AXI transaction. This proves the wrapper correctly blocks the processor for the full ReRAM write latency.
 2. **Non-Blocking Register Read:** In contrast, when the testbench initiates a read from the register region (e.g., ‘0x104’) by asserting S_AXI_ARVALID (5), the wrapper responds immediately. The S_AXI_RDATA and S_AXI_RVALID signals (6) are asserted on the very next clock cycle. This demonstrates that register-space accesses are non-blocking, providing immediate status to the processor as intended.

5.3.5. L0 Cell-Level Testbench Analysis

This testbench instantiates the `reram_cell_simple` module and performs directed tests to verify its core state machine, delay enforcement, and error logic.

The waveform in Figure 12 demonstrates the latency and endurance tests.

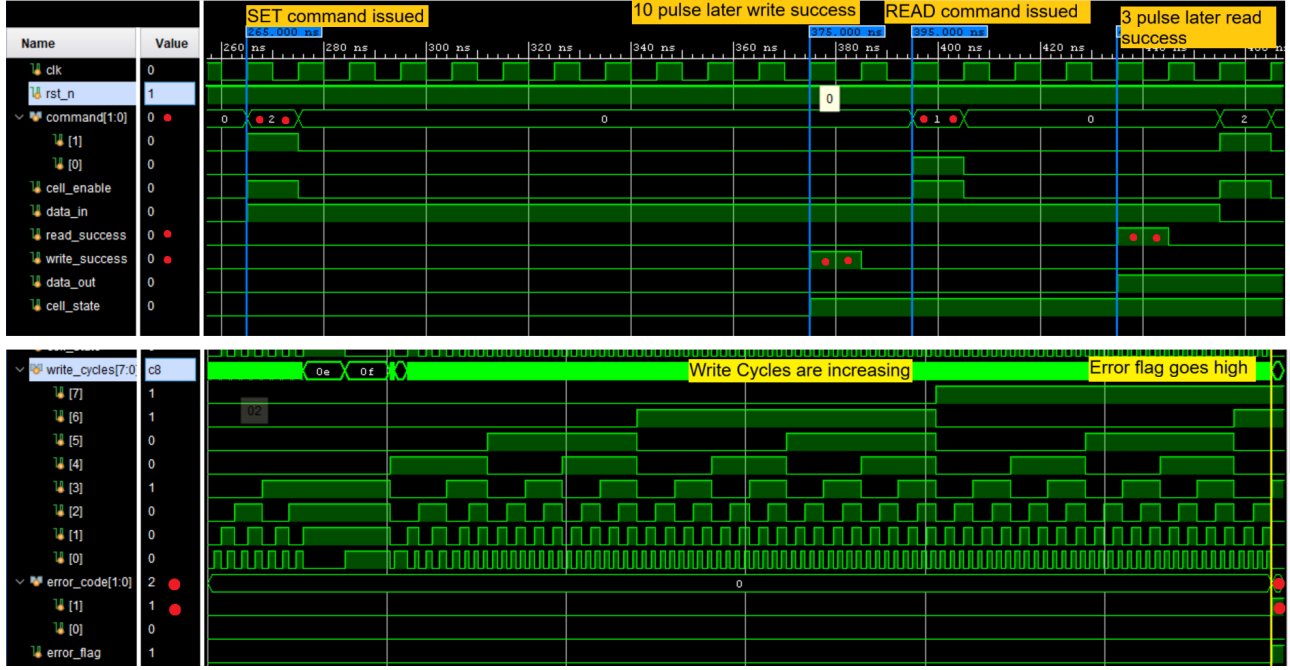


Figure 12: Waveform for `reram_cell_simple`. (1) A SET command is issued, and `write_success` pulses exactly 10 cycles later. (2) A READ command is issued, and `read_success` pulses 3 cycles later. (3) The `write_counter` is shown incrementing, eventually leading to the `ERR_WORN_OUT` flag (4) being asserted.

5.3.6. L3 AXI-Level Testbench

The most critical simulation verifies the top-level `reram_axilite_wrapper`. A SystemVerilog testbench was used to generate standard AXI-Lite read and write transactions.

5.3.7. Simulation Results and Waveform Analysis

The simulation console output confirms the sequence of operations, showing the testbench writing and then reading back the same value.

Listing 3: Simulation console output for AXI testbench.

```
ReRAM AXI-Lite Wrapper Testbench
=====
[0] Resetting AXI Wrapper and Core...
[65000] Reset complete

--- Phase 1: Basic Memory Read/Write Operations (Mem Space 0x000) ---
[65000] AXI Write: 0xdeadbeef to Addr 0x000
[235000] AXI Write: 0xcafebabe to Addr 0x004
[405000] AXI Write: 0x12345678 to Addr 0x03c
[575000] AXI Read: from Addr 0x000
[675000] AXI Read Data: 0xdeadbeef
[PASS] Data verification passed at 0x000
[675000] AXI Read: from Addr 0x004
[775000] AXI Read Data: 0xcafebabe
[PASS] Data verification passed at 0x004
```

```

[775000] AXI Read: from Addr 0x03c
[875000] AXI Read Data: 0x12345678
[PASS] Data verification passed at 0x03c
Phase 1 completed with 0 errors

--- Phase 2: Read Status Registers (Reg Space 0x100) ---
[875000] AXI Read: from Addr 0x100
[915000] AXI Read Data: 0x00000000
[915000] AXI Read: from Addr 0x104
[955000] AXI Read Data: 0x00000003
[PASS] Data verification passed at 0x104
[955000] AXI Read: from Addr 0x108
[995000] AXI Read Data: 0x00000003
[995000] AXI Read: from Addr 0x10c
[1035000] AXI Read Data: 0x00000000
[PASS] Data verification passed at 0x10c
Phase 2 completed with 0 errors

```

The waveform in Figure 13 provides a detailed view of the crucial **blocking write operation**.

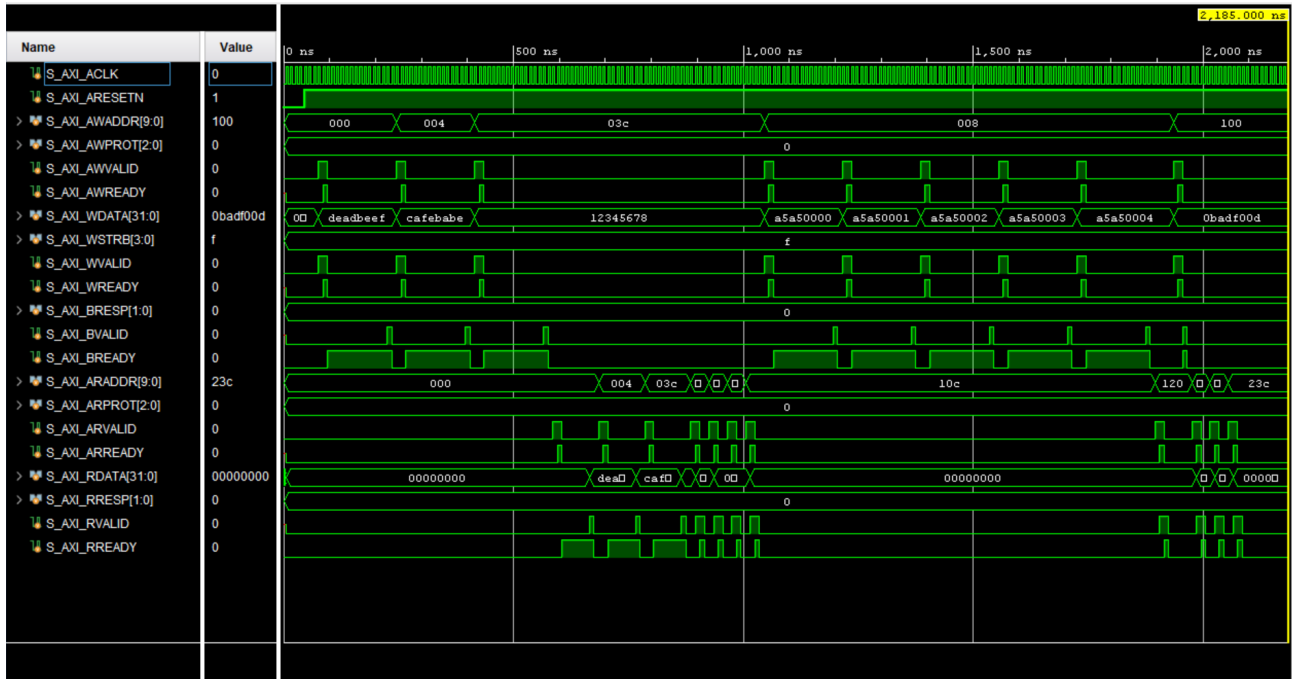


Figure 13: AXI blocking write transaction. (1) The testbench asserts `S_AXI_AWVALID` (address) and `S_AXI_WVALID` (data). (2) The wrapper latches the command and asserts the internal `core_busy_o` signal. (3) Crucially, the wrapper **stalls the AXI bus** by holding `S_AXI_BVALID` low. (4) The internal ReRAM operation completes, pulsing `core_done_o`. (5) Only *after* `core_done_o` is seen does the wrapper complete the AXI transaction by asserting `S_AXI_BVALID`.

5.4. Implementation Results

The design was successfully synthesized, placed, and routed using the Vivado toolchain.

5.4.1. RTL and Synthesized Schematics

The RTL schematic provides a high-level, structural view of the design, confirming the hierarchy. See Figure 14

After synthesis, the logic is mapped to the target FPGA’s specific technology elements. See Figure 15

5.4.2. Resource Utilization

The post-synthesis resource utilization report quantifies the hardware cost of the ReRAM IP. The parameters for this implementation were `MEMORY_DEPTH = 16` and `DATA_WIDTH = 32`.

Table 5: Post-Synthesis Resource Utilization

Resource	Estimation	Available	Utilization %
LUT	34545	53200	64,93
LUTRAM	66	17400	0,38
FF	26062	106400	24,49
BUFG	1	32	3,13

Analysis

The post-synthesis results show a significant utilization of logic resources, particularly Look-Up Tables (LUTs) at 64.93%. The Flip-Flop (FF) usage at 24.49% is also substantial. This is an expected outcome of the chosen design.

The ReRAM cell’s state, along with the write counters for endurance tracking, are not stored in dedicated Block RAM (BRAM). Instead, they are intentionally implemented using standard logic fabric (Flip-Flops and LUTs). This design choice allows the IP to model the behavior of a memory array of $\text{MEMORY_DEPTH} \times \text{DATA_WIDTH}$ ($16 \times 32 = 512$ bits) entirely in logic, which explains the high FF count. The `reram_cell_simple` module’s FSM, delay counters, and error logic contribute to the high LUT usage.

The use of 66 LUTRAMs (Distributed RAM) suggests that the synthesis tool inferred small, local memories from the RTL, likely for parts of the register logic or data multiplexing. As expected, dedicated Block RAM (BRAM) utilization is zero, confirming the design’s ”logic-only” emulation approach.

5.4.3. Power Utilization

Power analysis was performed using the Vivado Power Analyzer (XPA) on the post-implementation design, using activity data from simulation.

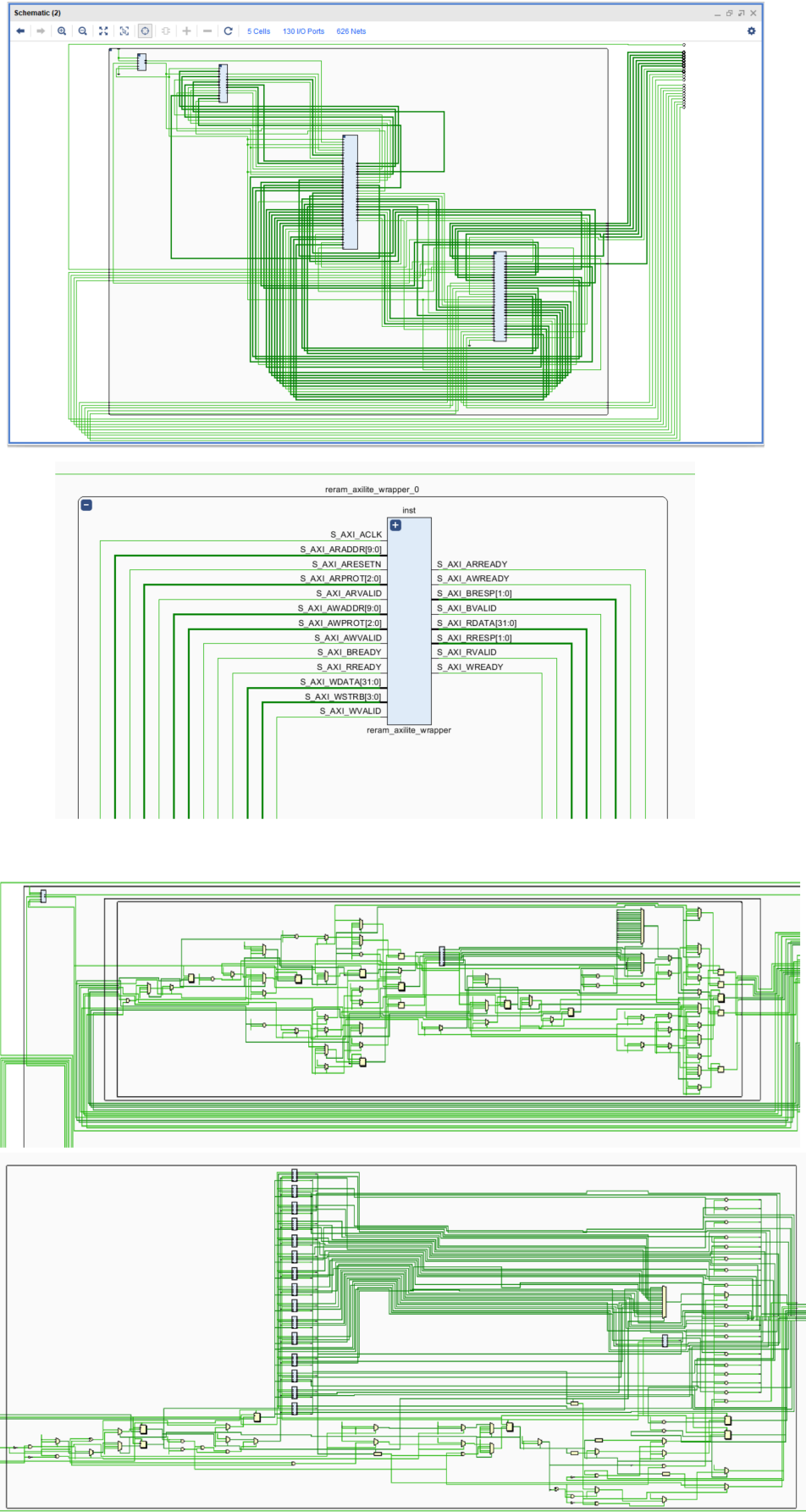


Figure 14: Elaborated RTL schematic of the `reram_axilite_wrapper`.

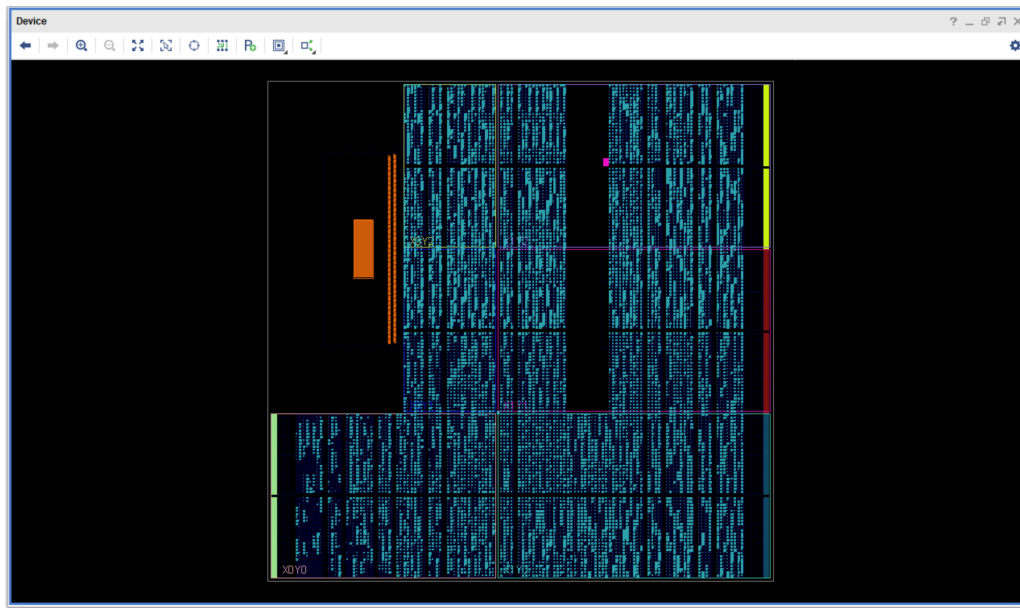


Figure 15: Synthesized schematic showing the design mapped to logic gates.

Summary

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power:	1.584 W
Design Power Budget:	Not Specified
Power Budget Margin:	N/A
Junction Temperature:	43.3°C
Thermal Margin:	41.7°C (3.5 W)
Effective θ_{JA}:	11.5°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Medium
Launch Power Constraint Advisor to find and fix invalid switching activity	

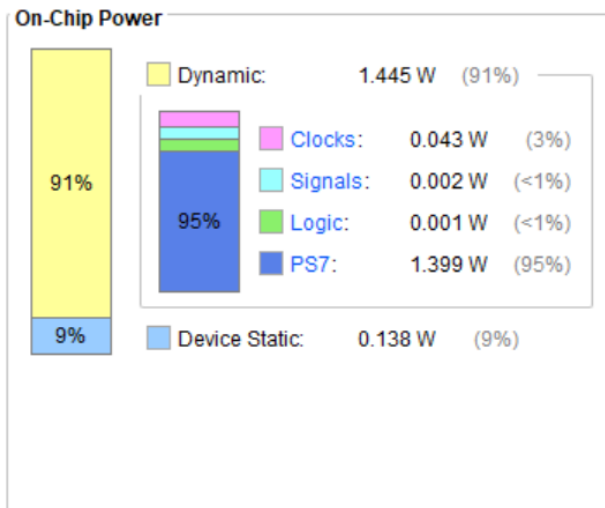


Figure 16: Power report summary from the Vivado Power Analyzer.

Analysis

The power report in Figure 16 shows a Total On-Chip Power of 1.584 W. This figure is overwhelmingly dominated by the Dynamic Power component, which accounts for 1.445 W (91%) of the total. The Device Static power is 0.138 W (9%), representing the baseline leakage for the device.

A critical insight from the report is that 95% of the dynamic power (1.399 W) is consumed by the PS7 (Processing System) block. This is the ARM processor, which is active and running the software.

In contrast, the power consumed by the ReRAM IP's hardware in the PL (Programmable Logic) is exceptionally low. The '**Logic**' component, which represents our synthesized design, consumes only 0.001 W. This confirms that while the overall system power is significant due to the processor, the

ReRAM controller IP itself is extremely power-efficient.

5.5. Reconfiguration to check parameterizability and scalability

To analyze the scalability of the design, a second configuration ("Small & Slow") was synthesized and compared against the baseline ("Standard"). Configuration 2 as shown in Figure 17 used a smaller memory size (MEMORY_DEPTH=8, DATA_WIDTH=16) and slower delay parameters.

Table 6: Resource Utilization Comparison

Resource	Config. 1 (16x32 "Standard")	Config. 2 (8x16 "Small")
LUT	34545	9108
LUTRAM	66	60
FF	26062	7031
BUFG	1	1

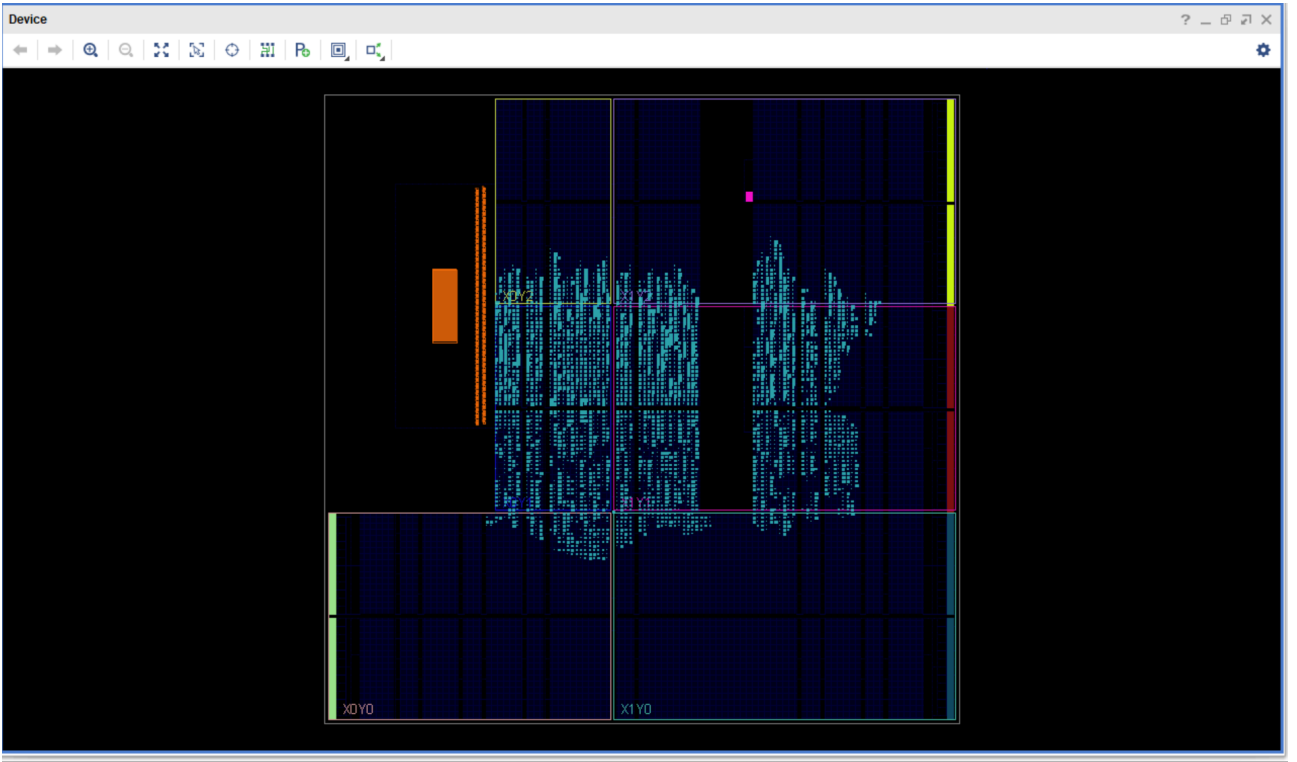


Figure 17: Vivado implementation results for Configuration 2 ("Small & Slow": 8×16 array). This screenshot confirms the data in Table 6, showing a significantly reduced resource footprint (9108 LUTs and 7031 FFs) compared to the baseline implementation.

Analysis

As shown in Table 6, the resource utilization for Configuration 2 is significantly lower, confirming the design's scalability.

The total number of emulated ReRAM cells was reduced from 512 (16×32) in Config. 1 to 128 (8×16) in Config. 2, a 75% reduction. This directly correlates to the drop in logic resources:

- **LUT utilization** decreased from 34545 to 9108, a reduction of **73.6%**.
- **FF utilization** decreased from 26062 to 7031, a reduction of **73.0%**.

This linear scaling confirms that the primary driver for resource cost is the **number of cells being emulated**. Each cell's FSM, endurance counter, and state register are implemented directly in logic (FFs and LUTs), so reducing the cell count provides a proportional reduction in resources.

As predicted, the delay parameters (SET_DELAY_CYCLES, etc.) had no noticeable impact on the utilization, as they only change a comparator value within existing logic.

5.5.1. Power Utilization Comparison

Power analysis was also performed on Configuration 2 ("Small & Slow") to observe the impact of the reduced logic footprint.

Table 7: Power Utilization Comparison

Metric	Config. 1 (16x32 "Standard")	Config. 2 (8x16 "Small")
Total On-Chip Power	1.584 W	1.554 W
- Dynamic Power	1.445 W	1.416 W
- Static Power	0.138 W	0.138 W
<i>Dynamic Breakdown</i>		
- PS7 (Processor)	1.399 W (95%)	1.399 W (97%)
- Clocks	0.043 W (3%)	0.015 W (1%)
- Logic (PL)	0.001 W (<1%)	0.001 W (<1%)
- Signals	0.002 W (<1%)	0.001 W (<1%)

Analysis

The power reports, summarized in Table 7, show that the Total On-Chip Power is almost identical for both configurations. The Static Power (0.138 W) is unchanged, as this is a characteristic of the device itself.

The vast majority of the power consumption in both scenarios is dynamic power (91%), and nearly all of that dynamic power is consumed by the PS7 (Processing System), which remained constant at 1.399 W. This is expected, as the ARM processor was running the same C-code application in both tests.

The only significant difference is in the power consumed by the Clocks, which dropped from 0.043 W to 0.015 W. This reduction is a direct result of the smaller design in Configuration 2. With fewer Flip-Flops (7,031 vs. 26,062) to drive, the clock tree in the PL is smaller and consumes less power.

This analysis confirms that the ReRAM IP's logic is extremely power-efficient, with its "Logic" power consumption (0.001 W) being negligible in both tests. The dominant factor in system power is the processor, not the size of the emulation logic.

5.6. Software Setup (Xilinx SDK / Vitis)

After exporting the hardware (bitstream and .xsa file) from Vivado, a software application was created in the Xilinx Vitis IDE (or older Xilinx SDK) to control the ReRAM IP from the ARM processor.

1. **Platform Project:** A new platform project was created based on the exported hardware.
2. **Board Support Package (BSP):** The BSP was generated, providing the low-level drivers and memory-map definitions.
3. **Application Project:** A new "Hello World" application was created to serve as the C-code driver.
4. **Bare-metal C-Code:** The C-code communicates with the ReRAM IP by writing directly to its assigned AXI memory addresses. The standard `xil_io.h` library provides functions (`Xil_Out32`, `Xil_In32`) for this purpose.

Note: The complete RTL for this module can be found in Listing C in Appendix 12 on page 72.

This C program, when run on the Zynq's processor, successfully controls the ReRAM IP as shown in Figure 18 , proving the entire hardware-software co-design is functional.

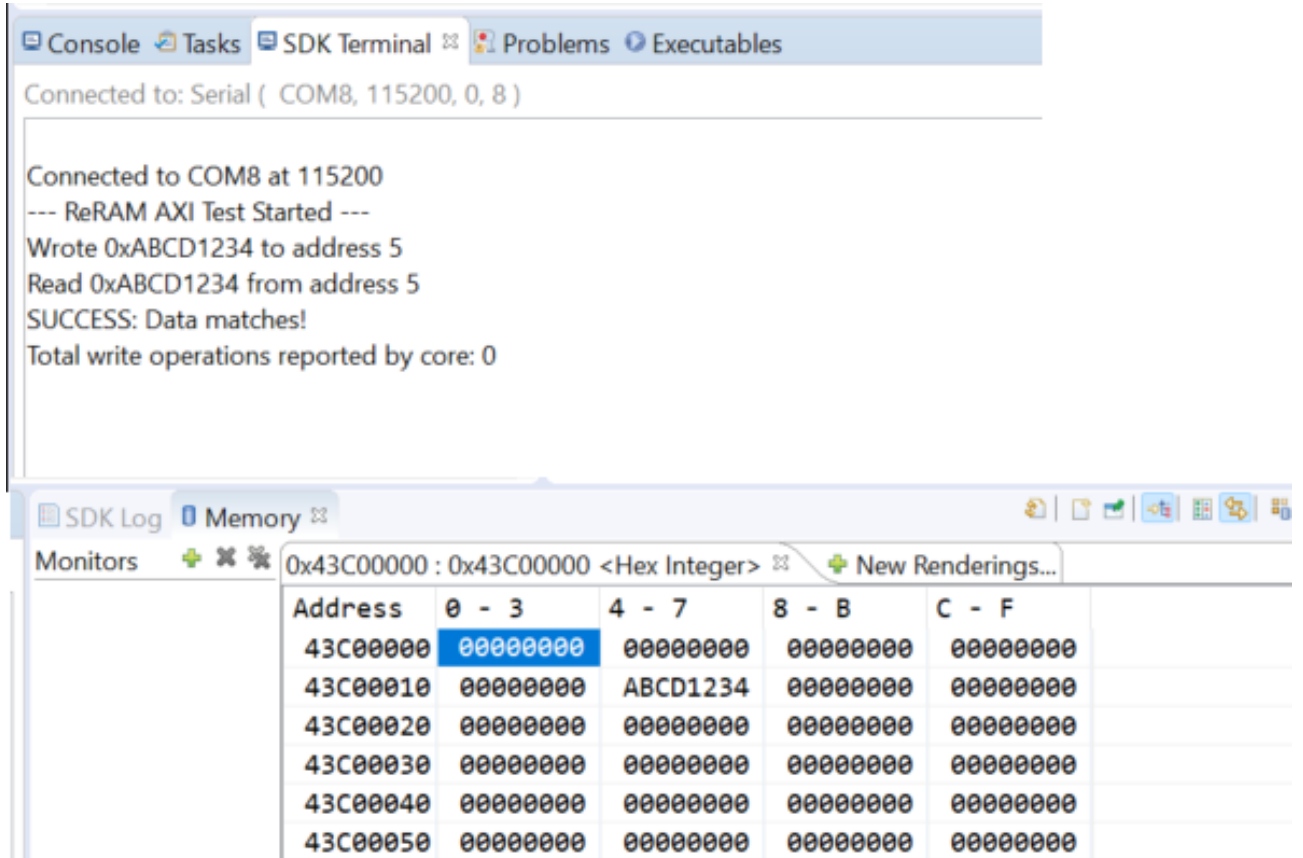


Figure 18: Hardware-software validation in the Xilinx Vitis/SDK environment. The bare-metal C application (Listing 12) runs on the ARM processor, interacting with the ReRAM IP over AXI. The UART console (top) displays the `printf` output, confirming a successful data read/write match, and the memory window (bottom) can be used to observe the IP's memory-mapped registers directly.

5.7. Chapter Summary

This section successfully validated the hierarchical ReRAM memory design. Functional simulation confirmed that the cell, word, controller, and AXI wrapper modules perform correctly, most notably enforcing the critical operation latencies and endurance limits.

The design was then synthesized and implemented on a ZyboZ7020 target, demonstrating a minimal hardware footprint with zero BRAM utilization, as intended by the design. Finally, a bare-metal C application confirmed that the AXI-Lite interface is fully functional, allowing a software application to control the ReRAM IP. The system is now a verified and validated platform for future experiments.

6. Conclusion

This thesis presented the design, implementation, and hardware-based evaluation of a parametrizable, behavioral ReRAM core on a Xilinx Zynq-7000 FPGA. The work successfully addressed the gap between high-level theoretical models and practical, system-level hardware by creating a reproducible emulation platform.

The core contribution is a four-layer, bottom-up SystemVerilog design that models ReRAM behavior in synthesizable hardware. At the lowest level (L0), the `reram_cell_simple` module successfully emulates the non-ideal, time-domain characteristics of ReRAM, including configurable write delays (SET/RESET) and a finite write endurance counter. This behavioral model was scaled into an addressable memory array (L1, L2) and abstracted by a top-level AXI-Lite wrapper (L3).

A key achievement of this architecture is its processor integration. The L3 wrapper provides a standard, memory-mapped interface to the ARM processing system. It translates simple AXI transactions into the time-based operations of the core by **stalling the AXI bus** (e.g., holding `S_AXI_BVALID` low), thus hiding the ReRAM's inherent latency from the software and simplifying driver development.

The entire design was successfully implemented on the Zybo Z7-7020 and validated with a bare-metal C application. This confirmed that the AXI communication and hardware-software co-design are fully functional. Implementation results characterized the resource cost of this behavioral-first approach, confirming the design's intentional use of logic (64.93% of LUTs) and **zero BRAM utilization**. Power analysis further revealed that the emulation logic's power consumption (0.001 W) is negligible compared to the Zynq's Processing System (1.399 W).

This work provides a validated, open-source foundation for future research in ReRAM-based architectures.

6.1. Future Work

While this thesis successfully delivered a validated ReRAM IP core, the platform opens several avenues for future research:

- **CNN Accelerator Integration:** The primary next step is to use this validated ReRAM IP as a foundational block for a Processing-in-Memory (PIM) accelerator. A new hierarchical layer (L4) could be developed to arrange the ReRAM cores into crossbar arrays for matrix-vector multiplication (MVM), a key operation in Convolutional Neural Networks (CNNs). The ARM processor would act as a controller, feeding input vectors and weights to the ReRAM array and initiating the MVM computation, leveraging the in-memory computing potential of ReRAM.
- **Advanced Cell Modeling:** The L0 cell model (`reram_cell_simple`) could be enhanced to support more complex, realistic behaviors. This includes modeling stochastic (probabilistic) switching, implementing write-verify schemes to tighten resistance distributions, and supporting intermediate resistance states for multi-level cell (MLC) capability. These features are critical for more accurate neuromorphic or analog computing simulations.
- **Operating System Integration:** The bare-metal C driver could be ported to a full PetaLinux kernel driver. This would allow the ReRAM array to be accessed as a standard character device

(e.g., `/dev/reram`) from a full-fledged Linux operating system, enabling more complex, multi-threaded applications and easier integration with existing software frameworks.

- **Scalability and Network-on-Chip (NoC):** To emulate larger, chip-scale ReRAM systems, multiple instances of the ReRAM IP core could be instantiated and connected via a Network-on-Chip (NoC). This would allow researchers to study the system-level performance implications of data locality, routing, and memory-access parallelism in a large-scale ReRAM-based architecture.

6.2. Resources Used

The development of this thesis was supported by a combination of modern software tools and foundational academic resources.

6.2.1. AI-Powered Development Tools

During the development of this thesis, AI-powered tools, including large language models, were utilized as a development aid. These tools were instrumental in accelerating the following tasks:

- **Code Generation and Debugging:** Assisting in the generation of boilerplate SystemVerilog for testbenches (e.g., clock generators, reset sequences, AXI transaction tasks). The tools also helped identify and resolve complex simulation errors, such as AXI protocol violations or combinatorial loops.
- **Documentation and Refinement:** Aiding in the drafting and iterative refinement of descriptive text, particularly in the literature review and implementation sections, to ensure clarity, conciseness, and correct technical terminology.
- **Conceptual Brainstorming:** Acting as a sounding board for architectural decisions, such as the design of the hierarchical FSMs (L0-L3) or the critical AXI bus-stalling mechanism in the L3 wrapper.

All final code, architectural designs, and written content were thoroughly reviewed, validated, and adapted by the author to ensure technical correctness, originality, and fulfillment of the project's objectives.

6.2.2. Textbooks and Academic Literature

The core theoretical foundation of this work was built upon established academic resources. Key texts on digital design, FPGA architecture, and non-volatile memory provided the essential principles:

- **Digital Design and FPGA Implementation:** Practical guidance for the SystemVerilog implementation was drawn from *Quick Start Guide to Verilog* by Brock J. LaMeres. This was supplemented by numerous YouTube video tutorials covering hardware implementation and practical demonstrations on the Zybo FPGA platform.
- **Academic Foundations:** Lecture notes on Computer Architecture Fundamentals from Prof. Dr.-Ing. Michael Wahl provided essential principles that informed the design of the memory hierarchy and processor interface.

- **Vendor Documentation:** Xilinx documentation (e.g., *AXI4-Lite Specification*, *Zynq-7000 Technical Reference Manual*) was the primary source for correctly implementing the L3 wrapper and integrating the IP with the Zynq Processing System.
- **Academic Papers:** Seminal papers on ReRAM modeling (as cited in the Literature Review) provided the basis for the behavioral characteristics (e.g., SET/RESET delays, endurance) implemented in the L0 cell.

Appendices

A. RTL Design Code (SystemVerilog)

A.1. L0: reram_cell_simple.sv

Listing 4: RTL for reram_cell_simple.sv

```

1  /*
2   * ReRAM Cell
3   *
4   * This module emulates a ReRAM cell focused purely on memory operations.
5   * This does not implement compute functionality
6   *
7   * Features:
8   * - Command-based interface with cell-level addressing
9   * - Read operation with configurable delay
10  * - Write operations (SET/RESET) with configurable delays
11  * - Binary resistance states (HRS/LRS)
12  * - Error reporting
13  * - Write cycle endurance tracking
14  *
15  * Interface:
16  * - Command input specifies operation type (NOP/READ/WRITE)
17  * - Cell enable provides individual cell addressing
18  * - Cell state output shows LRS=1, HRS=0 for intuitive logic
19  */
20
21  module reram_cell_simple #(
22      parameter int READ_DELAY_CYCLES = 3,    // Cycles for read operation
23      parameter int SET_DELAY_CYCLES = 10,    // Cycles for LRS transition
24      parameter int RESET_DELAY_CYCLES = 10,  // Cycles for HRS transition
25      parameter int MAX_WRITE_CYCLES = 255    // Maximum write endurance (8-bit)
26  ) (
27      input  logic      clk,
28      input  logic      rst_n,
29
30      // Command interface
31      input  logic [1:0] command,    // 00=NOP, 01=READ, 10=write
32      input  logic      cell_enable, // Cell addressing enable
33      input  logic      data_in,     // 1-bit data input for write
34
35      // Memory status outputs
36      output logic      read_success, // Read completed (1 cycle pulse)
37      output logic      write_success, // Write completed (1 cycle pulse)
38      output logic      data_out,     // 1-bit data output
39      output logic      cell_state,   // Cell state: 1=LRS, 0=HRS
40      output logic [7:0] write_cycles, // Total write cycles performed (8-bit)
41
42      // Error reporting
43      output logic [1:0] error_code,  // 00=no error, 01=busy, 10=worn out
44      output logic      error_flag    // Error occurred flag

```

```

45 );
46
47 // Command definitions
48 localparam [1:0] CMD_NOP    = 2'b00; // No operation
49 localparam [1:0] CMD_READ  = 2'b01; // Read operation
50 localparam [1:0] CMD_WRITE = 2'b10; // Write operation
51
52 // Error code definitions
53 localparam [1:0] ERR_NONE    = 2'b00; // No error
54 localparam [1:0] ERR_BUSY    = 2'b01; // Cell busy
55 localparam [1:0] ERR_WORN_OUT = 2'b10; // Cell worn out
56
57 typedef enum logic [1:0] {
58     IDLE,
59     READING,
60     WRITING_SET,      // Transitioning to LRS (lower resistance)
61     WRITING_RESET     // Transitioning to HRS (higher resistance)
62 } cell_state_t;
63
64 // Internal registers
65 cell_state_t current_state, next_state;
66 logic [31:0] delay_counter;
67 logic [7:0]  write_counter;      // 8-bit write counter
68 logic        internal_cell_state; // 1 = LRS, 0 = HRS (same as output)
69 logic        internal_data_out;
70
71 // Command processing
72 logic        active_read_cmd, active_write_cmd;
73
74 // Operation tracking
75 logic [31:0] target_delay, next_target_delay;
76 logic operation_complete;
77
78 // Success pulse generation
79 logic read_success_reg, read_success_next;
80 logic write_success_reg, write_success_next;
81
82 // Error tracking
83 logic [1:0] current_error_code;
84 logic current_error_flag;
85
86 //=====
87 // Command Processing
88 //=====
89
90 // Only respond to commands when cell is enabled
91 always_comb begin
92     active_read_cmd  = cell_enable && (command == CMD_READ);
93     active_write_cmd = cell_enable && (command == CMD_WRITE);
94 end
95
96 //=====

```

```

97 // State Machine Logic
98 //=====
99
100 always_comb begin
101     next_state = current_state;
102     operation_complete = 1'b0;
103     next_target_delay = 0;
104     current_error_code = ERR_NONE;
105     current_error_flag = 1'b0;
106
107     case (current_state)
108     IDLE: begin
109         // Check for worn out condition first
110         if (write_counter >= MAX_WRITE_CYCLES) begin
111             current_error_code = ERR_WORN_OUT;
112             current_error_flag = 1'b1;
113         end else if (active_read_cmd || active_write_cmd) begin
114             // Check for simultaneous operations (busy condition)
115             if (active_read_cmd && active_write_cmd) begin
116                 current_error_code = ERR_BUSY;
117                 current_error_flag = 1'b1;
118             end else if (active_read_cmd) begin
119                 next_state = READING;
120                 next_target_delay = READ_DELAY_CYCLES;
121             end else if (active_write_cmd && write_counter < MAX_WRITE_CYCLES)
122                 begin
123                     if (data_in == 1'b1) begin
124                         next_state = WRITING_SET; // Switch to LRS
125                         next_target_delay = SET_DELAY_CYCLES;
126                     end else begin
127                         next_state = WRITING_RESET; // Switch to HRS
128                         next_target_delay = RESET_DELAY_CYCLES;
129                     end
130                 end
131             end
132         end
133
134     READING, WRITING_SET, WRITING_RESET: begin
135         // Cell is busy - check if someone is trying to start new operations
136         if (active_read_cmd || active_write_cmd) begin
137             current_error_code = ERR_BUSY;
138             current_error_flag = 1'b1;
139         end
140
141         // Handle state transitions based on delays
142         case (current_state)
143         READING: begin
144             if (delay_counter >= target_delay) begin
145                 next_state = IDLE;
146                 operation_complete = 1'b1;
147             end
148         end
149     end
150 end

```



```

148         WRITING_SET, WRITING_RESET: begin
149             if (delay_counter >= target_delay) begin
150                 next_state = IDLE;
151                 operation_complete = 1'b1;
152             end
153         end
154     endcase
155 end
156
157     default: begin
158         next_state = IDLE;
159     end
160 endcase
161 end
162
163 //=====
164 // Sequential Logic
165 //=====
166
167 always_ff @(posedge clk or negedge rst_n) begin
168     if (!rst_n) begin
169         current_state <= IDLE;
170         delay_counter <= 0;
171         write_counter <= 0;
172         internal_cell_state <= 1'b0;    // Start in HRS (cell_state = 0)
173         internal_data_out <= 1'b0;
174         read_success_reg <= 1'b0;
175         write_success_reg <= 1'b0;
176         target_delay <= 0;
177     end else begin
178         current_state <= next_state;
179         read_success_reg <= read_success_next;
180         write_success_reg <= write_success_next;
181
182         // Delay counter management
183         if (current_state == IDLE || (current_state != next_state)) begin
184             delay_counter <= 1;
185             target_delay <= next_target_delay;
186         end else begin
187             delay_counter <= delay_counter + 1;
188         end
189
190         // Handle operation completion
191         if (operation_complete) begin
192             case (current_state)
193                 READING: begin
194                     internal_data_out <= internal_cell_state; // Output cell state
195                                     directly
196                 end
197                 WRITING_SET: begin
198                     internal_cell_state <= 1'b1; // Switch to LRS (cell_state = 1)
199                     // Increment with overflow protection

```

```

199         if (write_counter < 8'hFF) begin
200             write_counter <= write_counter + 1;
201         end
202     end
203     WRITING_RESET: begin
204         internal_cell_state <= 1'b0; // Switch to HRS (cell_state = 0)
205         // Increment with overflow protection
206         if (write_counter < 8'hFF) begin
207             write_counter <= write_counter + 1;
208         end
209     end
210 endcase
211 end
212 end
213 end
214
215 //=====
216 // Success Pulse Generation
217 //=====
218
219 always_comb begin
220     read_success_next = 1'b0;
221     write_success_next = 1'b0;
222
223     if (operation_complete) begin
224         case (current_state)
225             READING: begin
226                 read_success_next = 1'b1;
227             end
228             WRITING_SET, WRITING_RESET: begin
229                 write_success_next = 1'b1;
230             end
231         endcase
232     end
233 end
234
235 //=====
236 // Output Assignments
237 //=====
238
239 // Memory operation signals
240 assign read_success = read_success_reg;
241 assign write_success = write_success_reg;
242
243 // Memory data outputs
244 assign data_out = internal_data_out;
245 assign cell_state = internal_cell_state;
246 assign write_cycles = write_counter;
247
248 // Error reporting
249 assign error_code = current_error_code;
250 assign error_flag = current_error_flag;

```

```

251
252 endmodule

```

A.2. L1: reram_word_array_simple.sv

Listing 5: RTL for reram_word_array_simple.sv

```

1  /*
2  * Simple ReRAM Word Array - Memory Only
3  *
4  * This module instantiates WORD_WIDTH ReRAM cells to create a single word
5  * of memory with standard read/write operations only. No compute functionality.
6  */
7  module reram_word_array_simple #(
8      parameter int WORD_WIDTH = 32,          // Bits per word
9      parameter int READ_DELAY_CYCLES = 3,    // Cell read delay
10     parameter int SET_DELAY_CYCLES = 10,     // Cell SET delay
11     parameter int RESET_DELAY_CYCLES = 10,   // Cell RESET delay
12     parameter int MAX_WRITE_CYCLES = 200     // Cell endurance limit
13 ) (
14     input  logic      clk,
15     input  logic      rst_n,
16
17     // Command-based memory interface
18     input  logic [1:0] command,              // 00=NOP, 01=READ, 10=WRITE
19     input  logic      word_enable,          // Word addressing enable
20     input  logic [WORD_WIDTH-1:0] write_data, // Word data to write
21     output logic [WORD_WIDTH-1:0] read_data,  // Word data read
22     output logic      operation_complete,    // Memory op done
23
24     // Status and error reporting
25     output logic      word_busy,             // Any cell is busy
26     output logic [WORD_WIDTH-1:0] cell_error_flags, // Per-cell error flags
27     output logic      word_error,           // Any cell has error
28     output logic [7:0] total_write_cycles   // Max of all cell write cycles
29 );
30
31 // ... (localparams and internal signals) ...
32
33 //=====
34 // ReRAM Cell Array Instantiation
35 //=====
36
37 genvar i;
38 generate
39     for (i = 0; i < WORD_WIDTH; i++) begin : cell_instances
40         reram_cell_simple #(
41             .READ_DELAY_CYCLES(READ_DELAY_CYCLES),
42             .SET_DELAY_CYCLES(SET_DELAY_CYCLES),
43             .RESET_DELAY_CYCLES(RESET_DELAY_CYCLES),
44             .MAX_WRITE_CYCLES(MAX_WRITE_CYCLES)
45         ) cell_inst (

```

```

46         .clk(clk),
47         .rst_n(rst_n),
48         .command(cell_command_reg),
49         .cell_enable(cell_enables_reg[i]),
50         .data_in(cell_data_in[i]),
51
52         // ... (outputs) ...
53         .read_success(cell_read_success[i]),
54         .write_success(cell_write_success[i]),
55         .data_out(cell_data_out[i]),
56         .cell_state(cell_states[i]),
57         .write_cycles(cell_write_cycles[i]),
58         .error_code(cell_error_codes[i]),
59         .error_flag(cell_error_flag[i])
60     );
61     end
62 endgenerate
63
64 // ... (Simplified Word-Level State Machine) ...
65
66 // ... (Status Aggregation Logic) ...
67 always_comb begin
68     // The word is busy when not in IDLE state
69     any_cell_busy = (current_word_state != WORD_IDLE);
70
71     // Check completion of operations - all cells either succeed OR error
72     all_read_complete = &(cell_read_success | cell_error_flag);
73     all_write_complete = &(cell_write_success | cell_error_flag);
74
75     // Aggregate error flags
76     word_error = |cell_error_flag;
77 end
78
79 // ... (Sequential Logic for command pulsing) ...
80
81 // ... (Output Assignments) ...
82
83 endmodule

```

A.3. L2: reram_memory_controller_blocking.sv

Listing 6: RTL for reram_memory_controller_blocking.sv

```

1  /*
2   * Simple Blocking ReRAM Memory Controller
3   *
4   * This is a straightforward memory controller that provides a simple blocking
5   * interface to ReRAM memory.
6   */
7  module reram_memory_controller_blocking #(
8      parameter int MEMORY_DEPTH = 16,
9      parameter int DATA_WIDTH = 32,

```

```

10     parameter int ADDR_WIDTH = $clog2(MEMORY_DEPTH),
11     // ... (delay parameters) ...
12     parameter int MAX_WRITE_CYCLES = 200
13 ) (
14     input  logic          clk,
15     input  logic          rst_n,
16
17     // Simple Memory Interface
18     input  logic [ADDR_WIDTH-1:0]  addr_i,
19     input  logic [DATA_WIDTH-1:0]  data_i,
20     output logic [DATA_WIDTH-1:0]  data_o,
21     input  logic                  read_i,
22     input  logic                  write_i,
23     output logic                  done_o,
24     output logic                  error_o,
25     output logic                  busy_o,
26
27     // ... (status outputs) ...
28     // ... (flattened monitoring outputs) ...
29     output logic [MEMORY_DEPTH*8-1:0]  word_write_cycles_flat,
30     output logic [MEMORY_DEPTH*DATA_WIDTH-1:0]  word_cell_errors_flat
31 );
32
33 // ... (localparams, internal signals, controller FSM typedef) ...
34
35 //=====
36 // Word Array Instantiation
37 //=====
38
39 genvar i;
40 generate
41     for (i = 0; i < MEMORY_DEPTH; i++) begin : word_arrays
42         reram_word_array_simple #(
43             .WORD_WIDTH(DATA_WIDTH),
44             /* ... delay params ... */
45             .MAX_WRITE_CYCLES(MAX_WRITE_CYCLES)
46         ) word_array_inst (
47             .clk(clk),
48             .rst_n(rst_n),
49             .command(word_command_reg),
50             .word_enable(word_enables[i]),
51             .write_data(word_write_data),
52             .read_data(word_read_data[i]),
53             .operation_complete(word_operation_complete[i]),
54
55             // ... (status outputs) ...
56             .word_busy(word_busy[i]),
57             .word_error(word_error[i]),
58             .cell_error_flags(word_cell_error_flags[i]),
59             .total_write_cycles(word_total_write_cycles[i])
60         );
61     end

```

```

62     endgenerate
63
64     //=====
65     // Address Decoding
66     //=====
67
68     always_comb begin
69         valid_address = (current_addr < MEMORY_DEPTH);
70
71         // Generate word enables only when actively sending a command
72         word_enables = '0;
73         if (valid_address && operation_in_progress && (word_command_reg != CMD_NOP))
74             begin
75                 word_enables[current_addr] = 1'b1;
76             end
77
78         //=====
79         // State Machine
80         //=====
81
82         always_comb begin
83             next_state = state;
84
85             case (state)
86                 IDLE: begin
87                     if (read_i && !write_i) begin
88                         next_state = READING;
89                     end else if (write_i && !read_i) begin
90                         next_state = WRITING;
91                     end else if (read_i && write_i) begin
92                         next_state = ERROR_STATE; // Both asserted - invalid command
93                     end
94                 end
95
96                 READING: begin
97                     if (valid_address && word_operation_complete[current_addr]) begin
98                         next_state = IDLE;
99                     end else if (!valid_address) begin
100                         next_state = IDLE; // Invalid address, return to idle
101                     end
102                 end
103
104                 WRITING: begin
105                     if (valid_address && word_operation_complete[current_addr]) begin
106                         next_state = IDLE;
107                     end else if (!valid_address) begin
108                         next_state = IDLE; // Invalid address, return to idle
109                     end
110                 end
111
112                 ERROR_STATE: begin

```

```

113         // Return to IDLE after one cycle (immediate error completion)
114         next_state = IDLE;
115     end
116
117     default: next_state = IDLE;
118 endcase
119 end
120
121 // ... (Output logic, Sequential logic, Output flattening) ...
122
123 endmodule

```

A.4. L3: reram_axilite_wrapper.sv

Listing 7: RTL for reram_axilite_wrapper.sv

```

1 // =====
2 // ReRAM AXI-Lite Wrapper (Vivado 2018.1 friendly)
3 //
4 // CORRECTIONS APPLIED:
5 // 1. Flattened 2D array ports from DUT to 1D vectors to fix 2018.1 parser bug.
6 // 2. Fixed "Multiple Driver" error by driving core_addr_i combinationaly.
7 // 3. Fixed "Read Latch" bug by adding axi_rdata_reg.
8 // =====
9 module reram_axilite_wrapper #(
10     // ReRAM core
11     parameter integer MEMORY_DEPTH      = 16,
12     parameter integer DATA_WIDTH       = 32,
13     parameter integer CORE_ADDR_WIDTH   = $clog2(MEMORY_DEPTH),
14
15     // AXI-Lite
16     parameter integer C_S_AXI_DATA_WIDTH = 32,
17     parameter integer C_S_AXI_ADDR_WIDTH = 10 // 1024B region
18 ) (
19     // AXI-Lite Global
20     input  logic      S_AXI_ACLK,
21     input  logic      S_AXI_ARESETN,
22
23     // ... (AXI interface ports: AW, W, B, AR, R) ...
24     // AXI-Lite Write Address
25     input  logic [C_S_AXI_ADDR_WIDTH-1:0] S_AXI_AWADDR,
26     input  logic [2:0]                     S_AXI_AWPROT,
27     input  logic                           S_AXI_AWVALID,
28     output logic                           S_AXI_AWREADY,
29     // ... etc ...
30     // AXI-Lite Read Data
31     output logic [C_S_AXI_DATA_WIDTH-1:0] S_AXI_RDATA,
32     output logic [1:0]                     S_AXI_RRESP,
33     output logic                           S_AXI_RVALID,
34     input  logic                           S_AXI_RREADY
35 );
36

```

```

37 // -----
38 // Address map (byte addressing)
39 // -----
40 localparam integer ADDR_MEM_BASE = 10'h000;
41 localparam integer ADDR_MEM_END = ADDR_MEM_BASE + (MEMORY_DEPTH*4) - 1;
42
43 localparam integer ADDR_REG_BASE = 10'h100;
44 localparam integer ADDR_TOTAL_READS = ADDR_REG_BASE + 10'h00; // 0x100
45 localparam integer ADDR_TOTAL_WRITES = ADDR_REG_BASE + 10'h04; // 0x104
46 // ... (rest of address map) ...
47
48 // ... (ReRAM Core Wires) ...
49
50 // Core instance
51 reram_memory_controller_blocking #(
52     .MEMORY_DEPTH (MEMORY_DEPTH),
53     .ADDR_WIDTH (CORE_ADDR_WIDTH),
54     .DATA_WIDTH (DATA_WIDTH)
55 ) u_core (
56     .clk (S_AXI_ACLK),
57     .rst_n (S_AXI_ARESETN),
58     .addr_i (core_addr_i),
59     .data_i (core_data_i),
60     .data_o (core_data_o),
61     .read_i (core_read_i),
62     .write_i (core_write_i),
63     .done_o (core_done_o),
64     .error_o (core_error_o),
65     .busy_o (core_busy_o),
66     // ... (monitoring ports) ...
67 );
68
69 // ... (Unflatten monitoring buses) ...
70
71 // -----
72 // Write FSM
73 // -----
74 always_ff @(posedge S_AXI_ACLK) begin
75     if (!S_AXI_ARESETN) begin
76         // ... (reset logic) ...
77     end else begin
78         case (wr_state)
79             WR_IDLE: begin
80                 if (S_AXI_AWVALID && S_AXI_WVALID) begin
81                     // ... (latch address/data) ...
82                     wr_state <= WR_WAIT_CORE;
83                     if ((S_AXI_AWADDR >= ADDR_MEM_BASE) && (S_AXI_AWADDR <=
84                         ADDR_MEM_END))
85                         core_write_i <= 1'b1; // pulse
86                 end
87             end

```



```

88         WR_WAIT_CORE: begin
89             if ((axi_awaddr >= ADDR_MEM_BASE) && (axi_awaddr <= ADDR_MEM_END))
90                 begin
91                     if (core_done_o) begin // <-- BLOCKING wait
92                         axi_bvalid <= 1'b1;
93                         wr_state <= WR_RESP;
94                     end
95                     end else begin
96                         axi_bvalid <= 1'b1; // Non-blocking for regs
97                         wr_state <= WR_RESP;
98                     end
99                 end
100             WR_RESP: begin
101                 // ... (handshake logic) ...
102             end
103         endcase
104     end
105 end
106
107 // -----
108 // Read FSM
109 // -----
110 always_ff @(posedge S_AXI_ACLK) begin
111     if (!S_AXI_ARESETN) begin
112         // ... (reset logic) ...
113     end else begin
114         case (rd_state)
115             RD_IDLE: begin
116                 if (S_AXI_ARVALID) begin
117                     // ... (latch address) ...
118                     if ((S_AXI_ARADDR >= ADDR_MEM_BASE) && (S_AXI_ARADDR <=
119                         ADDR_MEM_END)) begin
120                         core_read_i <= 1'b1; // pulse
121                         rd_state <= RD_WAIT_CORE;
122                     end else begin
123                         rd_state <= RD_RESP; // register read is immediate
124                         axi_rvalid <= 1'b1;
125                         axi_rdata_reg <= axi_rdata_mux_out; // Latch mux data
126                     end
127                 end
128             end
129             RD_WAIT_CORE: begin
130                 if (core_done_o) begin // <-- BLOCKING wait
131                     axi_rvalid <= 1'b1;
132                     rd_state <= RD_RESP;
133                     axi_rdata_reg <= core_data_o; // Latch core data
134                 end
135             end
136             RD_RESP: begin

```

```

138             // ... (handshake logic) ...
139         end
140     endcase
141 end
142 end
143
144 // -----
145 // Read Data Mux (combinational, for registers)
146 // -----
147 always_comb begin
148     axi_rdata_mux_out = '0; // Default
149
150     if ((axi_araddr >= ADDR_CELL_ERRORS_BASE) && (axi_araddr <= ADDR_CELL_ERRORS_END)
151         ) begin
152         // ...
153     end
154     else begin
155         unique case (axi_araddr)
156             ADDR_TOTAL_READS:    axi_rdata_mux_out = core_total_reads;
157             ADDR_TOTAL_WRITES:   axi_rdata_mux_out = core_total_writes;
158             ADDR_WORD_ERRORS:    axi_rdata_mux_out = {{...}, core_word_errors};
159             // ... (rest of mux) ...
160             default:             axi_rdata_mux_out = '0;
161         endcase
162     end
163 end
164
165 // ... (AXI output assignments) ...
166 endmodule

```

B. Verification Testbenches (SystemVerilog)

B.1. L0 Testbench (tb_reram_cell_simple.sv)

Listing 8: L0 Testbench for reram_cell_simple.

```

1  /*
2   * Simple ReRAM Cell Testbench
3   *
4   * This testbench validates the simplified ReRAM cell that focuses on
5   * memory operations only. Tests read/write functionality, timing,
6   * error conditions, and endurance tracking.
7   */
8  module tb_reram_cell_simple;
9
10     // Test parameters
11     parameter int CLK_PERIOD = 10;
12     parameter int READ_DELAY = 3;
13     parameter int SET_DELAY = 10;
14     parameter int RESET_DELAY = 10;

```

```

15     parameter int MAX_WRITES = 200;
16
17     // ... (DUT signals) ...
18     logic clk;
19     logic rst_n;
20     logic [1:0] command;
21     logic cell_enable;
22     logic data_in;
23     logic read_success;
24     logic write_success;
25     logic data_out;
26     logic cell_state;
27     logic [7:0] write_cycles;
28     logic [1:0] error_code;
29     logic error_flag;
30
31     // ... (DUT Instantiation) ...
32     reram_cell_simple #(
33         .READ_DELAY_CYCLES(READ_DELAY),
34         .SET_DELAY_CYCLES(SET_DELAY),
35         .RESET_DELAY_CYCLES(RESET_DELAY),
36         .MAX_WRITE_CYCLES(MAX_WRITES)
37     ) dut (
38         /* ... connections ... */
39     );
40
41     // ... (Clock Generation) ...
42
43     // ... (Test Tasks: reset_system, write_set,
44     //      write_reset, read_cell, verify_value, test_timing) ...
45
46     // --- Main Test Sequence ---
47     initial begin
48         $display("--- Phase 1: Basic SET/RESET/READ Operations ---");
49         write_reset();
50         verify_value(1'b0);
51         write_set();
52         verify_value(1'b1);
53
54         $display("\n--- Phase 3: Timing Validation ---");
55         test_timing("READ", READ_DELAY);
56         test_timing("SET", SET_DELAY);
57
58         $display("\n--- Phase 6: Endurance Tracking ---");
59         logic [7:0] initial_count, final_count;
60         initial_count = write_cycles;
61         for (int i = 0; i < 20; i++) begin
62             if (i % 2 == 0) write_set();
63             else write_reset();
64         end
65         final_count = write_cycles;
66         $display("Write count progression: %0d -> %0d", initial_count, final_count);

```

```

67
68     $display("\n--- Phase 7: Endurance Limit Testing (DESTRUCTIVE) ---");
69     while (write_cycles < MAX_WRITES && !error_flag) begin
70         /* ... write repeatedly ... */
71     end
72     $display("Reached write limit. Current count: %0d", write_cycles);
73
74     // ... (Test summary and $finish) ...
75 end
76 endmodule

```

B.2. L1 Testbench (tb_reram_word_array_simple.sv)

Listing 9: L1 Testbench for reram_word_array_simple.

```

1  /*
2   * Simple ReRAM Word Array Testbench
3   *
4   * Validates a simplified 32-cell word array (reads/writes, integrity, timing).
5   */
6  module tb_reram_word_array_simple;
7
8      // Test parameters
9      parameter int CLK_PERIOD = 10;
10     parameter int WORD_WIDTH = 32;
11     parameter int READ_DELAY = 3;
12     parameter int SET_DELAY = 10;
13     parameter int RESET_DELAY = 10;
14     parameter int MAX_ENDURANCE_CYCLES = 200;
15
16     // ... (DUT signals) ...
17     logic clk;
18     logic rst_n;
19     logic [1:0] command;
20     logic word_enable;
21     logic [WORD_WIDTH-1:0] write_data;
22     logic [WORD_WIDTH-1:0] read_data;
23     logic operation_complete;
24     logic word_busy;
25     logic word_error;
26     logic [WORD_WIDTH-1:0] cell_error_flags;
27     logic [7:0] total_write_cycles;
28
29     // ... (DUT Instantiation) ...
30     reram_word_array_simple #(
31         .WORD_WIDTH(WORD_WIDTH),
32         .READ_DELAY_CYCLES(READ_DELAY),
33         /* ... other parameters ... */
34         .MAX_WRITE_CYCLES(MAX_ENDURANCE_CYCLES)
35     ) dut (
36         /* ... connections ... */
37     );

```

```

38
39 // ... (Clock Generation) ...
40
41 // ... (Test Tasks: reset_system, write_word,
42 //     read_word, verify_word, check_cell_health) ...
43
44 // --- Main Test Sequence ---
45 initial begin
46     $display("--- Phase 1: Basic Word Read/Write Operations ---");
47     write_word(32'hDEADBEEF); verify_word(32'hDEADBEEF);
48
49     $display("\n--- Phase 2: Bit Pattern Testing ---");
50     test_pattern(32'h00000000, "All Zeros");
51     test_pattern(32'hFFFFFFFF, "All Ones");
52
53     $display("\n--- Phase 6: Endurance Counter Tracking ---");
54     logic [7:0] writes_before, writes_after;
55     writes_before = total_write_cycles;
56     for (int e = 0; e < 5; e++) begin
57         write_word(32'h12345678 + e);
58     end
59     writes_after = total_write_cycles;
60     $display("Total write cycles progression: %0d -> %0d",
61         writes_before, writes_after);
62
63     $display("\n--- Phase 9: Endurance Limit Testing ---");
64     int remaining_to_limit = MAX_ENDURANCE_CYCLES - total_write_cycles;
65     for (int n = 0; n < remaining_to_limit + 5; n++) begin // +5 to go over limit
66         write_word(32'hDEADBEEF + n);
67         if (word_error) begin
68             $display("[ENDURANCE LIMIT HIT] at DUT_cycles=%0d", total_write_cycles);
69             break;
70         end
71     end
72
73     // ... (Test summary and $finish) ...
74 end
75 endmodule

```

B.3. L2 Testbench (tb_reram_memory_controller_blocking.sv)

Listing 10: L2 Testbench for reram_memory_controller_blocking.

```

1  /*
2   * Simple Testbench for Blocking ReRAM Memory Controller
3   *
4   * Validates the blocking memory controller with straightforward
5   * read/write operations, similar to traditional SRAM testing.
6   */
7  module tb_reram_memory_controller_blocking;
8
9     // Test parameters

```

```

10  parameter int CLK_PERIOD = 10;
11  parameter int ADDR_WIDTH = 4; // 16 words
12  parameter int DATA_WIDTH = 32;
13  parameter int MEMORY_DEPTH = 16;
14
15  // ... (DUT signals) ...
16  logic clk;
17  logic rst_n;
18  logic [ADDR_WIDTH-1:0] addr_i;
19  logic [DATA_WIDTH-1:0] data_i;
20  logic [DATA_WIDTH-1:0] data_o;
21  logic read_i;
22  logic write_i;
23  logic done_o;
24  logic error_o;
25  logic busy_o;
26
27  // ... (DUT Instantiation) ...
28  reram_memory_controller_blocking #(
29      .MEMORY_DEPTH(MEMORY_DEPTH),
30      .ADDR_WIDTH(ADDR_WIDTH),
31      .DATA_WIDTH(DATA_WIDTH),
32      .MAX_WRITE_CYCLES(200)
33  ) dut (
34      /* ... connections ... */
35  );
36
37  // ... (Clock Generation) ...
38
39  // Task: Write data to address
40  task automatic write_memory(input [ADDR_WIDTH-1:0] address,
41                             input [DATA_WIDTH-1:0] data);
42  begin
43      @(posedge clk);
44      addr_i = address;
45      data_i = data;
46      write_i = 1;
47      read_i = 0;
48
49      @(posedge clk);
50      write_i = 0;
51
52      // Wait for completion
53      wait(done_o);
54      @(posedge clk);
55  end
56  endtask
57
58  // ... (Other tasks: read_memory, write_read_verify, wear_out_address) ...
59
60  // --- Main Test Sequence ---
61  initial begin

```

```

62     $display("---- Phase 1: Basic Read/Write Operations ----");
63     write_read_verify(4'h0, 32'hDEADBEEF);
64     write_read_verify(4'h1, 32'hCAFEBABE);
65     write_read_verify(4'hF, 32'hA5A5A5A5); // End of memory
66
67     $display("\n--- Phase 6: Endurance Counter Testing ----");
68     logic [7:0] initial_cycles1, initial_cycles2;
69     initial_cycles1 = dut.word_write_cycles[4'h0];
70     initial_cycles2 = dut.word_write_cycles[4'h1];
71
72     $display("Performing 10 writes to address 0x0...");
73     for (int i = 0; i < 10; i++) begin
74         write_memory(4'h0, 32'h12345678 + i);
75     end
76     // ... (check counter for 0x0 incremented by 10) ...
77
78     $display("Performing 5 writes to address 0x1...");
79     for (int i = all; i < 5; i++) begin
80         write_memory(4'h1, 32'hABCDEF00 + i);
81     end
82     // ... (check counter for 0x1 incremented by 5) ...
83
84     // ... (Test summary and $finish) ...
85 end
86 endmodule

```

B.4. L3 Testbench (tb_reram_axilite_wrapper.sv)

Listing 11: L3 Testbench for reram_axilite_wrapper.

```

1  /*
2   * Testbench for ReRAM AXI-Lite Wrapper
3   *
4   * Emulates an AXI Master to test the 'reram_axilite_wrapper'.
5   */
6  module tb_reram_axilite_wrapper;
7
8      // Test parameters
9      parameter int CLK_PERIOD = 10;
10     parameter int MEMORY_DEPTH = 16;
11     parameter int DATA_WIDTH = 32;
12     parameter int AXI_ADDR_WIDTH = 10;
13
14     // ... (AXI Interface Signals) ...
15     logic S_AXI_ACLK;
16     logic S_AXI_ARESETN;
17     logic [AXI_ADDR_WIDTH-1:0] S_AXI_AWADDR;
18     logic S_AXI_AWVALID;
19     logic S_AXI_AWREADY;
20     logic [DATA_WIDTH-1:0] S_AXI_WDATA;
21     logic S_AXI_WVALID;
22     logic S_AXI_WREADY;

```

```

23     logic [1:0] S_AXI_BRESP;
24     logic S_AXI_BVALID;
25     logic S_AXI_BREADY;
26     logic [AXI_ADDR_WIDTH-1:0] S_AXI_ARADDR;
27     logic S_AXI_ARVALID;
28     logic S_AXI_ARREADY;
29     logic [DATA_WIDTH-1:0] S_AXI_RDATA;
30     logic [1:0] S_AXI_RRESP;
31     logic S_AXI_RVALID;
32     logic S_AXI_RREADY;
33
34     // ... (DUT Instantiation) ...
35     reram_axilite_wrapper #(
36         .MEMORY_DEPTH(MEMORY_DEPTH),
37         .DATA_WIDTH(DATA_WIDTH),
38         .C_S_AXI_DATA_WIDTH(DATA_WIDTH),
39         .C_S_AXI_ADDR_WIDTH(AXI_ADDR_WIDTH)
40     ) dut (
41         /* ... connections ... */
42     );
43
44     // ... (Clock Generation) ...
45
46     // ... (Test Tasks: reset_system, axi_write, axi_read, verify_read) ...
47
48     // --- Main Test Sequence ---
49     initial begin
50         reset_system();
51
52         $display("\n--- Phase 1: Basic Memory Read/Write Operations ---");
53         axi_write(10'h000, 32'hDEADBEEF); // Write to word 0
54         axi_write(10'h004, 32'hCAFEFEBABE); // Write to word 1
55         verify_read(10'h000, 32'hDEADBEEF);
56         verify_read(10'h004, 32'hCAFEFEBABE);
57
58         $display("\n--- Phase 2: Read Status Registers ---");
59         // After 2 writes and 2 reads...
60         verify_read(10'h100, 32'd2); // ADDR_TOTAL_READS
61         verify_read(10'h104, 32'd2); // ADDR_TOTAL_WRITES
62
63         $display("\n--- Phase 4: Test Write to Read-Only Register ---");
64         axi_write(10'h100, 32'hBADFOOD); // Attempt to write to TOTAL_READS
65         verify_read(10'h100, 32'd2); // Verify value is unchanged
66
67         // ... (Test summary and $finish) ...
68     end
69 endmodule

```

C. Software Application Code (C)

Listing 12: Bare-metal C code for testing the ReRAM IP.

```

1  #include "xparameters.h"
2  #include "xil_io.h"
3  #include <stdio.h>
4
5  // Define base address from Vivado Address Editor
6  #define RERAM_BASE_ADDR XPAR_RERAM_AXILITE_WRAPPER_0_S00_AXI_BASEADDR
7
8  // Define register offsets from reram_axilite_wrapper
9  #define MEM_OFFSET(addr)  (RERAM_BASE_ADDR + (addr * 4))
10 #define TOTAL_WRITES_REG  (RERAM_BASE_ADDR + 0x104)
11
12 int main() {
13     u32 write_data = 0xABCD1234;
14     u32 read_data;
15     u32 write_count;
16
17     printf("---ReRAM_AXI_Test_Started---\n");
18
19     // 1. Write data to memory address 5
20     Xil_Out32(MEM_OFFSET(5), write_data);
21     printf("Wrote 0x%08X to address 5\n", write_data);
22
23     // 2. Read data back from memory address 5
24     read_data = Xil_In32(MEM_OFFSET(5));
25     printf("Read 0x%08X from address 5\n", read_data);
26
27     // 3. Check for match
28     if (read_data == write_data) {
29         printf("SUCCESS: Data matches!\n");
30     } else {
31         printf("ERROR: Data mismatch!\n");
32     }
33
34     // 4. Read the total_writes status register
35     write_count = Xil_In32(TOTAL_WRITES_REG);
36     printf("Total write operations reported by core: %d\n", write_count);
37
38     return 0;
39 }

```

References

- [1] H. Akinaga and H. Shima, “Resistive random access memory (reram) based on metal oxides”, *Proceedings of the IEEE*, vol. 98, no. 12, pp. 2237–2251, 2010.
- [2] S. Ambrogio, S. Balatti, A. Cubeta, A. Calderoni, N. Ramaswamy, and D. Ielmini, “Statistical fluctuations in hfox resistive-switching memory: Part i - set/reset variability”, *IEEE Transactions on Electron Devices*, vol. 61, no. 8, pp. 2912–2919, 2014.
- [3] T. S. Böske, J. Müller, D. Bräuhäus, U. Schröder, and U. Böttger, “Ferroelectricity in hafnium oxide thin films”, *Applied Physics Letters*, vol. 99, no. 10, p. 102903, Sep. 2011.
- [4] P. Carrasco and I. Vourkas, “Fpga implementation of an elementary reram memory control unit”, in *2024 20th International Conference on Synthesis, Modeling, Analysis and Simulation Methods and Applications to Circuit Design (SMACD)*, 2024, pp. 1–4.
- [5] P.-Y. Chen, B. Lin, I.-T. Wang, *et al.*, “Mitigating effects of non-ideal synaptic device characteristics for on-chip learning”, in *2015 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2015, pp. 194–199.
- [6] P.-Y. Chen, X. Peng, and S. Yu, “Neurosim+: An integrated device-to-algorithm framework for benchmarking synaptic devices and array architectures”, *2017 IEEE International Electron Devices Meeting (IEDM)*, pp. 6.1.1–6.1.4, 2017.
- [7] P.-Y. Chen and S. Yu, “Compact modeling of rram devices and its applications in 1t1r and 1s1r array design”, *IEEE Transactions on Electron Devices*, vol. 62, no. 12, pp. 4022–4028, 2015.
- [8] P. Chi, S. Li, C. Xu, *et al.*, “Prime: A novel processing-in-memory architecture for neural network computation in reram-based main memory”, *SIGARCH Comput. Archit. News*, vol. 44, no. 3, pp. 27–39, Jun. 2016.
- [9] L. Chua, “Memristor-the missing circuit element”, *IEEE Transactions on Circuit Theory*, vol. 18, no. 5, pp. 507–519, 1971.
- [10] B. Gao, J. F. Kang, Y. S. Chen, *et al.*, “Oxide-based rram: Unified microscopic principle for both unipolar and bipolar switching”, in *2011 International Electron Devices Meeting*, 2011, pp. 17.4.1–17.4.4.
- [11] K. Garelo, F. Yasin, and G. S. Kar, “Spin-orbit torque mram for ultrafast embedded memories: From fundamentals to large scale technology integration”, in *2019 IEEE 11th International Memory Workshop (IMW)*, 2019, pp. 1–4.
- [12] D. Ielmini, “Modeling the universal set/reset characteristics of bipolar rram by field- and temperature-driven filament growth”, *IEEE Transactions on Electron Devices*, vol. 58, no. 12, pp. 4309–4317, 2011.
- [13] D. Ielmini, “Brain-inspired computing with resistive switching memory (rram): Devices, synapses and neural networks”, *Microelectronic Engineering*, vol. 190, pp. 44–53, 2018.
- [14] M. Kozicki, M. Park, and M. Mitkova, “Nanoscale memory elements based on solid-state electrolytes”, *IEEE Transactions on Nanotechnology*, vol. 4, no. 3, pp. 331–338, 2005.

- [15] S. Kvatinsky, E. G. Friedman, A. Kolodny, and U. C. Weiser, “Team: Threshold adaptive memristor model”, *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 60, no. 1, pp. 211–221, 2013.
- [16] S. Kvatinsky, M. Ramadan, E. G. Friedman, and A. Kolodny, “Vteam: A general model for voltage-controlled memristors”, *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 62, no. 8, pp. 786–790, 2015.
- [17] T. Luo, X. Wang, C. Qu, *et al.*, “An fpga-based hardware emulator for neuromorphic chip with rram”, *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 39, no. 2, pp. 438–450, 2020.
- [18] C. Mead, “Neuromorphic electronic systems”, *Proceedings of the IEEE*, vol. 78, no. 10, pp. 1629–1636, 1990.
- [19] S. Menzel, U. Böttger, and R. Waser, “Simulation of multilevel switching in electrochemical metallization memory cells”, *Journal of Applied Physics*, vol. 111, no. 1, p. 014501, Jan. 2012.
- [20] G. Pedretti and D. Ielmini, “In-memory computing with resistive memory circuits: Status and outlook”, *Electronics*, vol. 10, no. 9, 2021.
- [21] S. Raoux, W. Welnic, and D. Ielmini, “Phase change materials and their application to non-volatile memories”, *Chemical Reviews*, vol. 110, no. 1, pp. 240–267, 2010, PMID: 19715293.
- [22] A. Sawa, “Resistive switching in transition metal oxides”, *Materials Today*, vol. 11, no. 6, pp. 28–36, 2008.
- [23] J. F. Scott and C. A. P. de Araujo, “Ferroelectric memories”, *Science*, vol. 246, no. 4936, pp. 1400–1405, 1989.
- [24] A. Sebastian, M. Le Gallo, R. Khaddam-Aljameh, and E. Eleftheriou, “Memory devices and applications for in-memory computing”, *Nature Nanotechnology*, vol. 15, no. 7, pp. 529–544, 2020.
- [25] A. Shafiee, A. Nag, N. Muralimanohar, *et al.*, “Isaac: A convolutional neural network accelerator with in-situ analog arithmetic in crossbars”, in *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*, 2016, pp. 14–26.
- [26] I. Vourkas and G. C. Sirakoulis, *Memristor-Based Nanoelectronic Computing Circuits and Architectures*, Foreword by Leon Chua (Emergence, Complexity and Computation), 1st ed. Springer Cham, 2016.
- [27] Z. Wang, C. Li, P. Lin, *et al.*, “In situ training of feed-forward and recurrent convolutional memristor networks”, *Nature Machine Intelligence*, vol. 1, no. 9, pp. 434–442, 2019.
- [28] R. Waser and M. Aono, “Nanoionics-based resistive switching memories”, *Nature Materials*, vol. 6, no. 11, pp. 833–840, 2007.
- [29] W. A. Wulf and S. A. McKee, “Hitting the memory wall: Implications of the obvious”, *SIGARCH Comput. Archit. News*, vol. 23, no. 1, pp. 20–24, Mar. 1995.
- [30] L. Xia, B. Li, T. Tang, *et al.*, “Mnsim: Simulation platform for memristor-based neuromorphic computing system”, *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 37, no. 5, pp. 1009–1022, 2018.

- [31] J. J. Yang, M. D. Pickett, X. Li, D. A. Ohlberg, D. R. Stewart, and R. S. Williams, “Memristive switching mechanism for metal/oxide/metal nanodevices”, *Nature Nanotechnology*, vol. 3, no. 7, pp. 429–433, 2008.
- [32] S. Yu, “Neuro-inspired computing with emerging nonvolatile memorys”, *Proceedings of the IEEE*, vol. 106, no. 2, pp. 260–285, 2018.
- [33] S. Yu, Y. Wu, R. Jeyasingh, D. Kuzum, and H.-S. P. Wong, “An electronic synapse device based on metal oxide resistive switching memory for neuromorphic computation”, *IEEE Transactions on Electron Devices*, vol. 58, no. 8, pp. 2729–2737, 2011.